

Justify — Cross-Chain Bridge Architecture

How we build bridges for event / prediction markets across multiple blockchains

Inspired by the Chainlink CCIP architecture

Arc · Base · Ethereum · Polygon · BNB Smart Chain

PolyMarket Social ("Justify") — Bridge documentation set

Justify Bridge Architecture — documentation set

How Justify builds **cross-chain bridges** for its event / prediction markets across multiple blockchains (Arc, Base, Ethereum, Polygon, BSC), and how that bridge layer **is directly inspired by the Chainlink CCIP architecture**.

This set expands [§5 "Bridge-proxied markets"](#) of the Justify whitepaper from a one-page concept into a buildable design.

Acknowledgement of inspiration. *Justify's bridge layer is **explicitly and directly inspired by the architecture of Chainlink CCIP** (Cross-Chain Interoperability Protocol). Where Chainlink separates a chain-local Router from per-lane OnRamp/OffRamp contracts, carries messages over a Decentralized Oracle Network (DON), and guards the system with an independent Risk Management Network, Justify mirrors that separation of concerns in its own `IBridgeAdapter` / `MarketProxy` / home-market design. We do not reimplement Chainlink; we adopt its architectural pattern and use CCIP itself as the first production transport adapter.*

Reading order

#	Document	What it covers
00	overview.md	The home/proxy model, the Chainlink-inspired layering, glossary
01	bridge-adapter-interface.md	The <code>IBridgeAdapter</code> seam: message shape, lifecycle, guarantees
02	chainlink-ccip-adapter.md	The CCIP adapter and the explicit mapping to Chainlink's architecture
03	marketproxy-and-home-market.md	<code>MarketProxy</code> , wrapped receipts, lock/mint, home-market wiring
04	per-chain-bridge-playbook.md	How to stand up a bridge per chain; lanes, tokens, CCTP for USDC legs
05	cross-chain-resolution.md	Settlement propagation; CCIP + CRE; one resolution settles all chains
06	security-and-risk-management.md	Bridge trust, allowlists, exposure caps, circuit breaker (Chainlink RMN parallel)

Status

Designed (forward-looking). The MVP is single-chain; Phase 2 is multi-chain native; Phase 3 is this bridge layer. See the whitepaper roadmap (§8). Nothing here is deployed yet — these documents define the target so the contracts and adapters can be built against a frozen seam.

Relation to the rest of the repo

- **Whitepaper §5** — the concept this set expands.
- **`docs/delivery/cre-resolver-plan.md`** — the CRE resolver that, in the bridge phase, settles the home market once and propagates outward.
- **`docs/delivery/arc-layer-plan.md`** — the env-driven "chain is a parameter" property that makes per-chain deployment config, not a rewrite.
- **`contracts/src/`** — `MarketFactory` , `MarketAMM` , `PredictionMarket` , `OracleResolver` are the home-chain stack the proxies forward to.

Justify Bridge Architecture — Overview

Bridge documentation set — entry point

Version: 1.0 (design phase)

Date: 2026-06-13

Status: Designed, not implemented. This is the forward-looking target for Phase 3 of the Justify roadmap (see whitepaper §8). The MVP is single-chain; Phase 2 is multi-chain native; Phase 3 is this cross-chain bridge layer.

1. The problem: per-chain liquidity fragmentation

Justify deploys its prediction-market protocol natively to **Arc, Base, Ethereum, Polygon, and BNB Smart Chain** (see whitepaper §4). In the multi-chain native phase (Phase 2), each market lives on exactly one home chain:

- A market created on Base accepts trades only from Base users with Base USDC.
- An identical real-world event could spawn separate markets on Polygon and Ethereum, each with its own isolated AMM pool, its own price, and no way to arbitrage them.
- Users must bridge capital manually between chains to participate in markets on networks where they don't already hold collateral.

The result: **liquidity fragmentation by chain for the same event**. A market on a small chain has thin depth; a market on a large chain excludes users on other networks. The protocol's multi-chain reach becomes a liability — it splits, rather than unifies, the available capital.

This document describes how Justify's **cross-chain bridge layer** solves that fragmentation by proxying markets across chains, presenting one logical market per event with unified liquidity backed by a single AMM pool on a designated home chain.

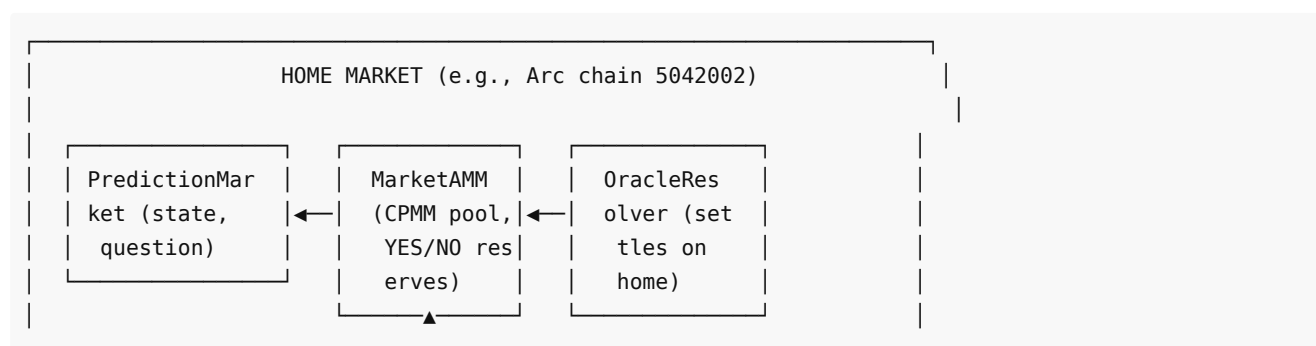
2. The solution: home-market / proxy-market model

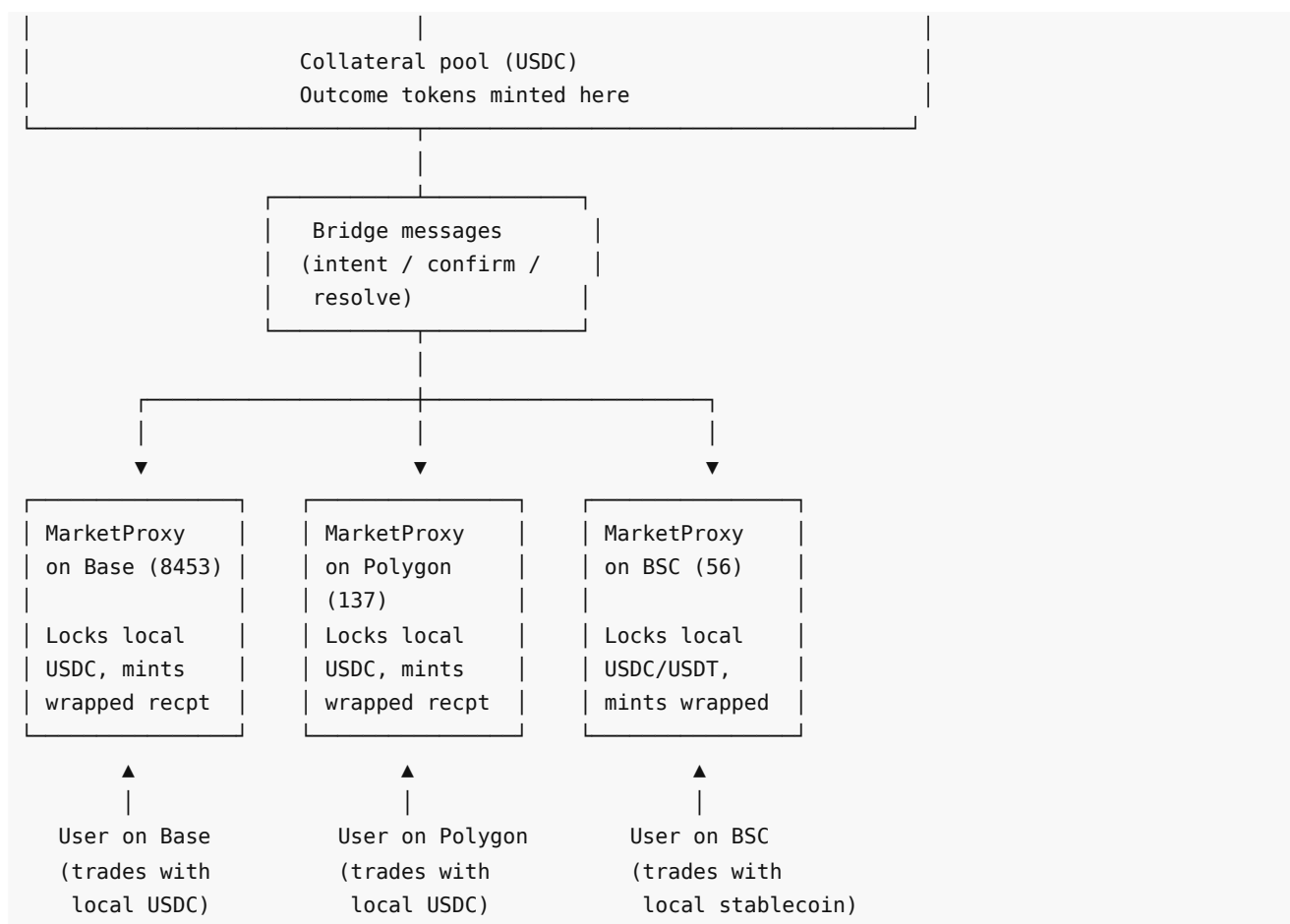
2.1 Conceptual model

A bridge-proxied market has:

1. **One home chain** — the chain where the market was created and where its `PredictionMarket`, `MarketAMM`, collateral pool, and `OracleResolver` actually live. This is the **source of truth** for prices, settlement, and state.
2. **Lightweight proxy contracts on every other supported chain** — a `MarketProxy` per remote chain that accepts local collateral (e.g., USDC on Polygon), locks it, emits a bridge message (a buy or sell *intent*) to the home market, receives a confirmation, and mints a **wrapped position receipt** representing the outcome tokens held on the home chain.
3. **A pluggable bridge adapter per route** — the transport layer that carries intents from a proxy to the home market and carries confirmations and resolutions back. The adapter is an interface (`IBridgeAdapter`); concrete implementations wrap Chainlink CCIP, Circle CCTP, LayerZero-class messaging, or other cross-chain transports.

2.2 ASCII diagram





2.3 Trade flow (proxy to home)

1. **User initiates a buy on a proxy chain.** Alice on Base calls `MarketProxy.buy(outcomeIndex=0, collateralAmount=100 USDC, minSharesOut=95)`.
2. **Proxy locks collateral.** The proxy transfers Alice's 100 USDC into escrow and records a pending intent.
3. **Intent bridged to home.** The proxy invokes its `IBridgeAdapter.sendBuyIntent()`, which encodes the message (marketId, user, outcomeIndex, amount, minShares) and transmits it via the underlying transport (e.g., Chainlink CCIP).
4. **Home market executes trade.** The home-chain bridge receiver decodes the intent and calls `MarketAMM.buy()` against the unified pool. The AMM calculates shares based on its CPMM reserves, mints outcome tokens to a bridge escrow, and emits a `TradeConfirmed` event.
5. **Confirmation bridged back to proxy.** The home adapter sends a confirmation message (intentId, sharesOut) back to the proxy chain.
6. **Proxy mints wrapped receipt.** The proxy receives the confirmation, verifies it, and mints a **wrapped position token** to Alice representing her claim on the outcome shares held on the home chain. Alice's 100 USDC is now locked on Base; her wrapped receipt entitles her to redeem for home-chain outcome tokens (or, after resolution, for collateral) via a bridge withdrawal.

2.4 Settlement flow (home to proxies)

1. **Home market resolves.** The `OracleResolver` on the home chain calls `PredictionMarket.resolve(winningOutcome=0)`. The market state transitions to `Resolved`.
2. **Resolution propagates outward.** The home bridge adapter emits a resolution message to every registered proxy: `MarketResolved(marketId, winningOutcome)`.
3. **Proxies mirror the resolution.** Each `MarketProxy` receives the resolution message, records the winning outcome, and transitions to a redeemable state.
4. **Users redeem locally.** Alice on Base calls `MarketProxy.redeem()`. The proxy verifies her wrapped receipt corresponds to the winning outcome, burns the receipt, and transfers the locked Base USDC (proportional to her shares) back to her. No second bridge hop — redemption happens on the chain where she entered.

3. Chainlink-inspired layering

Acknowledge the inspiration. Justify's bridge architecture is **directly inspired by the Chainlink Cross-Chain Interoperability Protocol (CCIP) architecture**. We adopt Chainlink's proven separation of concerns, layering strategy, and risk-management approach as the foundation of our design. We do **not** reimplement Chainlink; we use CCIP itself as the first production bridge adapter and mirror its architectural pattern for other transports.

3.1 The Chainlink CCIP model (reference)

Chainlink CCIP separates cross-chain messaging into four layers:

Chainlink CCIP layer	Role
Router (per chain)	The chain-local entry point. Users and applications send cross-chain messages to the Router, which forwards them to the appropriate lane-specific OnRamp.
OnRamp / OffRamp (per lane)	Per source→destination lane contracts. OnRamp on the source chain encodes and commits messages; OffRamp on the destination chain decodes and delivers them.
Decentralized Oracle Network	A committee of independent oracle nodes that observe committed messages on the source chain, reach consensus, and attest them on the destination chain's OffRamp.
Risk Management Network (RMN)	An independent monitoring layer that observes message flow and can halt the OffRamp (pause message execution) if anomalies are detected — a circuit breaker.

Key properties:

- **Separation of concerns:** chain-local routing, per-lane transport, consensus, and risk monitoring are independent layers with defined interfaces between them.
- **Pluggable lanes:** adding a new source→destination route is a deploy-and-register operation, not a protocol change.
- **Defense in depth:** the RMN provides an independent safety layer on top of the DON consensus.

3.2 How Justify maps Chainlink's layers

Justify adopts this pattern in its bridge design:

Justify bridge component	Chainlink CCIP analog	Role in Justify
MarketProxy (per chain)	Router	The chain-local entry point for bridged markets. Accepts user intents (buy/sell), locks collateral, emits events to the bridge adapter. One proxy contract per remote chain per bridged market.
IBridgeAdapter (interface)	OnRamp / OffRamp abstraction	The per-route transport seam. <code>sendBuyIntent()</code> / <code>receiveBuyConfirm()</code> / <code>sendResolution()</code> methods. Concrete adapters implement this for CCIP, CCTP, LayerZero, etc.
Concrete adapter (e.g., CCIP)	OnRamp / OffRamp + DON message transport	A Justify adapter wrapping Chainlink CCIP: encodes intents into CCIP messages on the source chain, relays via Chainlink's DON, decodes on the destination. Reuses CCIP's consensus and finality.
Per-route exposure caps + pause	Risk Management Network (RMN) circuit breaker	Justify's risk layer: each adapter has a per-route max exposure cap (total locked collateral) and a <code>pause()</code> function callable by protocol governance on anomaly detection. (See doc 06.)
Home-market settlement	Canonical state anchor	The home chain's <code>PredictionMarket</code> and <code>MarketAMM</code> are the single source of truth; proxies never resolve independently — they only mirror home state.

3.3 Why we adopt Chainlink's pattern

1. **Proven at scale.** CCIP's architecture has secured billions in cross-chain value. We inherit its design rigor rather than inventing our own untested model.
2. **Separation of transport from application logic.** `IBridgeAdapter` is a clean seam; swapping CCIP for LayerZero or Wormhole is a new adapter implementation, not a rewrite of `MarketProxy` or `MarketAMM`.
3. **Defense in depth.** The RMN-inspired exposure caps and circuit breaker give protocol governance a safety valve independent of any single bridge's security.
4. **Pluggable lanes.** Adding Base ↔ Polygon as a new route is a deploy operation (deploy adapters on both chains, register them with the proxies) with no changes to the factory or AMM contracts.
5. **Forward compatibility with decentralized resolution.** When Justify moves to automated oracle resolution (Phase 4, [whitepaper §8](#)), the resolution message is carried by the same adapter that carries trade intents — decentralizing resolution on the home chain decentralizes it everywhere at once.

3.4 Differences from Chainlink CCIP

Aspect	Chainlink CCIP	Justify bridge
Message types	Arbitrary data + token transfers	Three message types: buy intent, sell intent (future), resolution. Strongly typed per the <code>IBridgeAdapter</code> interface.
Finality	DON waits for probabilistic finality (e.g., 15 blocks on Ethereum) before attesting	Justify adapters inherit the finality guarantees of their underlying transport (CCIP: same as Chainlink; CCTP: Circle finality).
Token standard	CCIP has a native token-transfer standard with burn-and-mint or lock-and-mint token pools	Justify uses lock-and-mint for collateral (proxy locks USDC, home uses it to buy shares) and wrapped receipts for positions.
Governance	Chainlink DAO / node-operator set managed by Chainlink Labs	Justify protocol governance (initially multisig, later DAO) manages adapter allowlist and exposure caps per route.
Application-specific state	CCIP is general-purpose messaging	Justify messages are prediction-market-specific: they carry <code>marketId</code> , <code>outcomeIndex</code> , shares, oracle proofs.

4. Target chains and deployment scope

The bridge layer spans the same five networks as the multi-chain native phase (see [whitepaper §4.1](#)):

Network	Chain ID	Native collateral	Role in bridge architecture
Arc (Circle)	5042002	USDC — native gas <i>and</i> collateral asset	The stablecoin-native chain; markets created here use native USDC for gas and trading — the cleanest UX for dollar markets.
Base	8453	USDC	The social-consumer L2; low fees, the largest onchain-social user base — natural home for feed-driven prediction markets.
Ethereum	1	USDC / DAI	The settlement anchor and deepest capital base; canonical home for high-value markets and (likely) the bridge root.
Polygon PoS	137	USDC	The incumbent prediction-market ecosystem (Polymarket's home); cheapest entry for high-frequency retail flow.
BNB Smart Chain	56	USDC / USDT	The largest retail population in Asia and the global south; distribution at scale.

Every market designates one of these five as its **home chain** at creation time. The other four chains each receive a `MarketProxy` for that market if the creator opts in to cross-chain bridging. Initially, high-liquidity markets (seeded by the protocol or prominent creators) will be bridged; smaller markets may remain single-chain to avoid bridge overhead.

5. Glossary

Term	Definition
Home chain / home market	The blockchain where a market's <code>PredictionMarket</code> , <code>MarketAMM</code> , collateral pool, and <code>OracleResolver</code> are deployed and where settlement actually happens. The single source of truth for the market.
Proxy chain / proxy market	A blockchain other than the home chain where a lightweight <code>MarketProxy</code> contract accepts local collateral and forwards trade intents to the home market via a bridge.
Lane / route	A unidirectional source→destination bridge path, e.g., "Base → Arc" or "Arc → Polygon." Each route has its own <code>IBridgeAdapter</code> deployment and its own exposure cap.
Bridge adapter	A contract (or set of contracts) implementing <code>IBridgeAdapter</code> for a specific transport (CCIP, CCTP, LayerZero). Encodes messages on the source, relays them, decodes on the destination.
Intent	A cross-chain message requesting a trade on the home market: <code>BuyIntent(marketId, user, outcomeIndex, collateralAmount, minSharesOut)</code> OR <code>SellIntent(...)</code> .
Wrapped position receipt	An ERC-20 or ERC-1155 token minted by <code>MarketProxy</code> on a proxy chain representing a user's claim on outcome shares held in escrow on the home chain.
Attestation	A cryptographic proof (signature, Merkle proof, or consensus certificate) carried in a bridge message verifying that the source-chain event (intent or resolution) is finalized.
Exposure cap	A per-route maximum for total locked collateral on a proxy chain for a given market or globally. A risk-management parameter enforced by the bridge adapter; acts as a circuit breaker.
Circuit breaker	A pause mechanism callable by protocol governance to halt new intents on a route (or all routes) without blocking redemptions. Inspired by Chainlink's Risk Management Network.

6. Forward references — the rest of this documentation set

The bridge architecture is detailed across seven documents (including this overview):

#	Document	What it covers
00	00-overview.md (this document)	The problem (liquidity fragmentation), the home/proxy model, the Chainlink-inspired layering, glossary.
01	01-bridge-adapter-interface.md	The <code>IBridgeAdapter</code> Solidity interface: message types, lifecycle, events, guarantees, and the seam between proxies and adapters.
02	02-chainlink-ccip-adapter.md	The CCIP adapter: explicit mapping to Chainlink's Router/OnRamp/OffRamp/DON, message encoding, fee handling, finality assumptions.
03	03-marketproxy-and-home-market.md	<code>MarketProxy</code> contract design, wrapped position receipts, lock/mint flow, home-market AMM wiring, state synchronization.
04	04-per-chain-bridge-playbook.md	Operational playbook: how to stand up a bridge per chain pair, deploy adapters, configure lanes, use CCTP for USDC legs.
05	05-cross-chain-resolution.md	Settlement propagation: how <code>OracleResolver.resolve()</code> on the home chain triggers resolution messages to all proxies via adapters.

06	06-security-and-risk-management.md	Bridge trust model, adapter allowlists, per-route exposure caps, circuit breaker, Chainlink RMN parallel, governance.
----	---	---

7. Status and integration with the Justify roadmap

Current phase (as of 2026-06-13): Phase 1 (Arc testnet deployment) is in progress. The MVP (Phase 0) is complete and runs on local Ganache.

Bridge phase: This documentation set describes **Phase 3** of the Justify roadmap (whitepaper §8). Nothing here is deployed yet. Phase 2 (multi-chain native — five independent deployments, no bridges) comes first.

Design freeze: These documents define the target architecture so that when Phase 3 begins, the `IBridgeAdapter` interface, `MarketProxy` contract, and per-chain adapter deployments can be built against a stable specification. The interfaces and message shapes described here are the **integration contract** between the bridge layer and the home-chain stack (`MarketFactory`, `MarketAMM`, `PredictionMarket`, `OracleResolver`).

Related documentation:

- **Whitepaper §5** (`documentation_justify_whitepaper/justify-whitepaper.md`) — the one-page concept this set expands.
- **CRE resolver plan** (`docs/delivery/cre-resolver-plan.md`) — the Chainlink Functions-based automated resolver that, in the bridge phase, settles the home market once and propagates outward.
- **Arc migration spec** (`docs/delivery/arc-layer-plan.md`) — the proof that "chain is a config parameter," validating the claim that per-chain deployment is operational, not architectural.
- **Home-chain contracts** (`contracts/src/`) — `MarketFactory.sol`, `MarketAMM.sol`, `PredictionMarket.sol`, `OracleResolver.sol` are the existing stack the proxies forward to.

8. Properties and trade-offs recap

Property	Consequence / design choice
Unified liquidity	Every chain's users deepen the <i>same</i> AMM pool; small chains get big-chain depth from day one. One price per outcome per event — no arbitrage, no cross-chain price divergence.
Single source of truth	Home market resolves once; proxies mirror that resolution. No proxy can disagree with home state — the <code>OracleResolver</code> on the home chain is the only authority.
Asynchronous fills	A bridged buy confirms in bridge time (CCIP: ~10 min, optimistic bridges: faster). UI quotes a <i>guaranteed-bounds</i> fill (max slippage envelope) at intent time; home AMM fills within it or refunds.
Bridge trust	A proxy position is as safe as $\min(\text{home chain security, bridge adapter security})$. Adapters are allowlisted per route by governance; per-route exposure caps limit blast radius.
Local-first routing	If a market's home chain <i>is</i> the user's chain, the proxy path is bypassed entirely — native speed, no bridge fees, no bridge risk. The feed automatically routes users to the cheapest path.
Pluggable transports	The <code>IBridgeAdapter</code> interface lets us swap CCIP for LayerZero, Wormhole, or native token bridges (CCTP for USDC) without changing <code>MarketProxy</code> or <code>MarketAMM</code> . Future-proof.
Progressive deployment	High-liquidity markets bridge first; long-tail markets can remain single-chain. The architecture supports both bridged and non-bridged markets in parallel.

9. Next steps

Readers new to the bridge layer should proceed in document order:

1. **01-bridge-adapter-interface.md** — understand the message types and interface.
2. **02-chainlink-ccip-adapter.md** — see the CCIP mapping concretely.
3. **03-marketproxy-and-home-market.md** — learn how proxies lock collateral and mint wrapped receipts.
4. **04-per-chain-bridge-playbook.md** — operational details for standing up a route.
5. **05-cross-chain-resolution.md** — how settlement propagates.
6. **06-security-and-risk-management.md** — risk controls and governance.

Engineers building the bridge layer should treat `IBridgeAdapter` (doc 01) as the frozen integration contract; concrete adapters (CCIP, CCTP, LayerZero) implement that interface.

End of overview. Proceed to `01-bridge-adapter-interface.md` for the message specification and interface definition.

IBridgeAdapter — The Cross-Chain Transport Seam

Document: 01-bridge-adapter-interface.md

Status: Designed (Phase 3 — bridge-proxied markets)

Related: [Whitepaper §5](#) | [02-chainlink-ccip-adapter.md](#) | [03-marketproxy-and-home-market.md](#)

1. Design principle — injected transport dependency

The `IBridgeAdapter` follows the **injected-dependency pattern** established throughout Justify's contract architecture. Just as `MarketFactory` accepts the collateral token as a constructor parameter (not a hard-coded address), and just as `MarketAMM` receives the fee treasury as a deployment argument, the bridge transport is **a parameter, not a commitment**.

```
// MarketFactory – collateral injected at deploy
constructor(
    address _acl,
    address _outcomeToken,
    address _collateral,          // ← injected dependency
    address _feeTreasury
) { ... }

// MarketProxy – bridge adapter injected at deploy
constructor(
    uint256 _homeMarketId,
    uint64 _homeChainSelector,
    address _bridgeAdapter,      // ← injected dependency
    address _collateral
) { ... }
```

Result: Justify can deploy `MarketProxy` contracts with different adapters for different routes:

- CCIP adapter for Arc ↔ Base (Chainlink CCIP messaging + Circle CCTP for USDC)
- LayerZero adapter for Base ↔ Polygon (LayerZero V2 messaging)
- Wormhole adapter for Ethereum ↔ BSC (Wormhole-class messaging)

One interface. Many transports. The bridge is **configuration**, not architecture.

2. Interface sketch — Chainlink CCIP-inspired

Justify's `IBridgeAdapter` is **directly inspired by Chainlink CCIP's messaging model**: a Router/Client pattern where senders construct typed message payloads, call a router that returns a message ID, and receivers implement a callback invoked by the off-ramp upon message delivery.

Chainlink CCIP's architecture:

- **Client.EVM2AnyMessage** — a typed struct carrying the destination chain selector, receiver address, data payload, and token transfer instructions.
- **IRouterClient.ccipSend** — the send function; returns a `bytes32 messageId`.
- **ccipReceive** — the callback interface; destination contracts implement `IAny2EVMMessageReceiver` and process incoming messages in `ccipReceive`.
- **Chain selectors** — 64-bit identifiers for each supported chain, decoupling the protocol from raw chain IDs.

Justify adopts the same structure:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;
```

```

/// @title IBridgeAdapter
/// @notice Cross-chain messaging interface for Justify bridge-proxied markets.
///         Inspired by Chainlink CCIP's Router/Client model: typed payloads,
///         destination chain selectors, message-ID returns, and receiver callbacks.
///
///         One interface, many implementations: CCIP, LayerZero-class, CCTP for
///         USDC value legs. The adapter is an injected dependency (constructor
///         parameter to MarketProxy), never compiled into the protocol.
interface IBridgeAdapter {
    /// =====
    /// Message payload types
    /// =====

    /// @notice A cross-chain BUY intent from a proxy chain to the home market.
    struct BuyIntent {
        uint256 marketId;           // Home market ID
        address trader;             // Trader address on proxy chain
        uint8 outcomeIndex;         // 0 = YES, 1 = NO
        uint256 collateralIn;       // Amount in collateral units (USDC 6-decimal)
        uint256 maxSlippage;        // Maximum acceptable slippage in basis points
        uint256 nonce;              // Per-trader nonce for ordering/idempotency
    }

    /// @notice A cross-chain SELL intent from a proxy chain to the home market.
    struct SellIntent {
        uint256 marketId;           // Home market ID
        address trader;             // Trader address on proxy chain
        uint8 outcomeIndex;         // 0 = YES, 1 = NO
        uint256 sharesIn;          // Shares to sell
        uint256 minPayout;          // Minimum acceptable payout (slippage guard)
        uint256 nonce;              // Per-trader nonce
    }

    /// @notice Confirmation of fill execution on the home market → proxy.
    struct FillConfirmation {
        bytes32 intentMessageId;    // The original intent's message ID
        uint256 marketId;
        address trader;
        uint8 outcomeIndex;
        uint256 sharesOut;          // For buy: shares minted; for sell: collateral out
        uint256 executionPrice;     // Actual price in collateral per share (6-decimal)
    }

    /// @notice Resolution broadcast from home market to all proxies.
    struct ResolutionBroadcast {
        uint256 marketId;
        uint8 winningOutcome;       // 0 = YES, 1 = NO, 255 = INVALID
        uint256 resolvedAt;         // Timestamp
    }

    /// @notice Refund notice when a fill fails (slippage exceeded, market closed).
    struct RefundNotice {
        bytes32 intentMessageId;    // The original intent's message ID
        uint256 marketId;
        address trader;
    }

```

```

        uint256 refundAmount;    // Collateral to refund on proxy chain
        uint8 reason;           // 1 = slippage, 2 = market closed, 3 = other
    }

    /// =====
    /// Send interface
    /// =====

    /// @notice Send a cross-chain message. Returns a unique message ID.
    /// @param destChainSelector 64-bit chain selector (Chainlink CCIP convention).
    /// @param receiver          Destination contract address.
    /// @param payload            ABI-encoded message (BuyIntent | SellIntent | etc.).
    /// @param gasLimit           Gas limit for execution on destination chain.
    /// @return messageId         Unique identifier for this message (tracking/idempotency).
    function sendMessage(
        uint64 destChainSelector,
        address receiver,
        bytes calldata payload,
        uint256 gasLimit
    ) external payable returns (bytes32 messageId);

    /// @notice Quote the fee (in native gas token or LINK) for a cross-chain send.
    /// @param destChainSelector Target chain.
    /// @param receiver          Destination contract.
    /// @param payload            Message payload.
    /// @param gasLimit           Execution gas limit.
    /// @return feeAmount         Fee in wei (native) or LINK, depending on adapter.
    function quoteFee(
        uint64 destChainSelector,
        address receiver,
        bytes calldata payload,
        uint256 gasLimit
    ) external view returns (uint256 feeAmount);

    /// =====
    /// Receive interface
    /// =====

    /// @notice Callback invoked by the bridge adapter when a message arrives.
    ///          Destination contracts (MarketProxy, home PredictionMarket) must
    ///          implement this and decode the payload into the appropriate struct.
    ///
    ///          Analogous to Chainlink CCIP's `ccipReceive`.
    /// @param sourceChainSelector Origin chain selector.
    /// @param sender              Sender contract address on origin chain.
    /// @param payload             ABI-encoded message.
    function receiveMessage(
        uint64 sourceChainSelector,
        address sender,
        bytes calldata payload
    ) external;

    /// =====
    /// Events
    /// =====

```

```

event MessageSent(
    bytes32 indexed messageId,
    uint64 indexed destChainSelector,
    address indexed receiver,
    bytes payload,
    uint256 fee
);

event MessageReceived(
    bytes32 indexed messageId,
    uint64 indexed sourceChainSelector,
    address indexed sender,
    bytes payload
);
}

```

3. Canonical message payloads

The five message types above establish the **cross-chain integration contract** between proxy chains and home markets. Each is ABI-encoded into the `payload` field of `sendMessage`.

3.1 BuyIntent

Field	Type	Purpose
marketId	uint256	Home market ID (unique within home chain's MarketFactory)
trader	address	Trader's address on the proxy chain (refunds/receipts go here)
outcomeIndex	uint8	0 = YES, 1 = NO
collateralIn	uint256	Collateral locked on proxy (USDC 6-decimal units)
maxSlippage	uint256	Slippage tolerance in basis points (e.g. 300 = 3%)
nonce	uint256	Per-trader nonce for message ordering and idempotency

Flow: MarketProxy locks the trader's collateral, sends a BuyIntent to the home market. The home MarketAMM fills it (or refunds if slippage exceeded).

3.2 SellIntent

Field	Type	Purpose
marketId	uint256	Home market ID
trader	address	Trader on proxy chain
outcomeIndex	uint8	0 = YES, 1 = NO
sharesIn	uint256	Wrapped shares to burn on proxy
minPayout	uint256	Minimum collateral payout (slippage guard)
nonce	uint256	Per-trader nonce

Flow: MarketProxy burns the trader's wrapped receipt, sends SellIntent. Home AMM sells the underlying shares, sends collateral back via FillConfirmation.

3.3 FillConfirmation

Field	Type	Purpose
intentMessageId	bytes32	Links confirmation to the original intent (idempotency key)
marketId	uint256	Home market ID
trader	address	Trader on proxy chain
outcomeIndex	uint8	0 = YES, 1 = NO
sharesOut	uint256	For buy: outcome shares received; for sell: collateral payout
executionPrice	uint256	Actual fill price (collateral per share, 6-decimal)

Flow: Home market → proxy. Proxy mints wrapped receipts (buy case) or unlocks refunded collateral (sell case).

3.4 ResolutionBroadcast

Field	Type	Purpose
marketId	uint256	Home market ID
winningOutcome	uint8	0 = YES, 1 = NO, 255 = INVALID (market refunds all positions)
resolvedAt	uint256	Unix timestamp of resolution (for dispute windows in future phases)

Flow: Home OracleResolver settles the market once; resolution is bridged to all proxies. Users redeem wrapped receipts for collateral on their local chain.

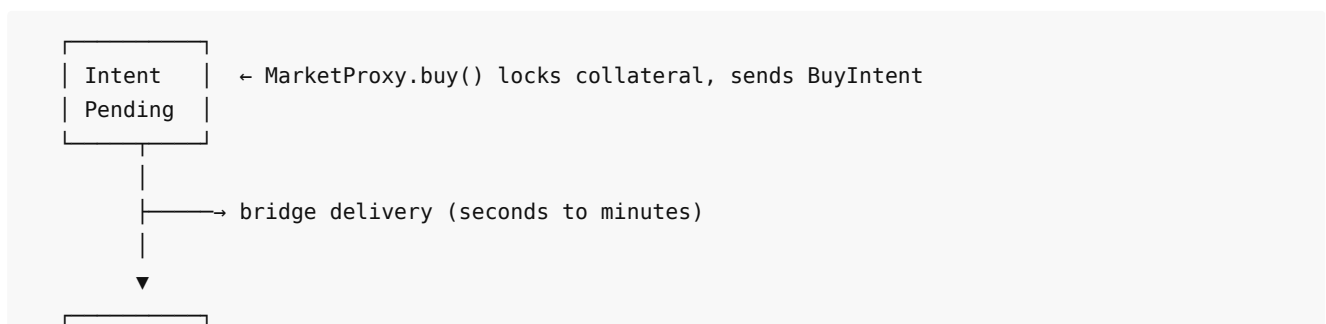
3.5 RefundNotice

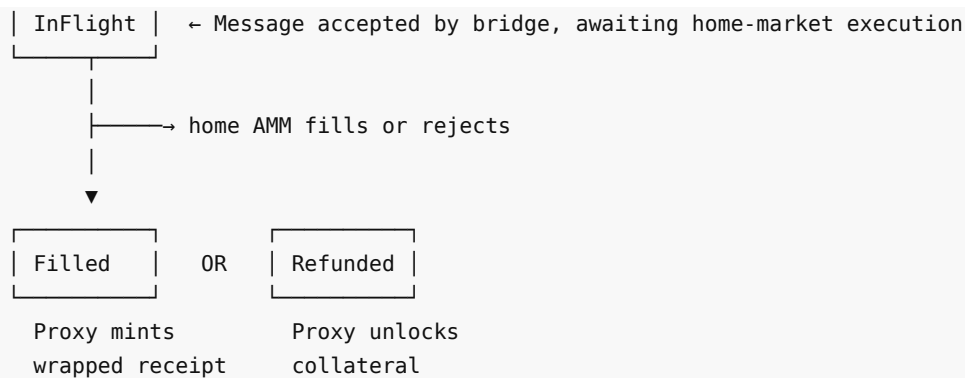
Field	Type	Purpose
intentMessageId	bytes32	Links refund to the original intent
marketId	uint256	Home market ID
trader	address	Trader on proxy chain
refundAmount	uint256	Collateral to unlock and return to trader
reason	uint8	1 = slippage exceeded, 2 = market closed, 3 = other failure

Flow: Home market → proxy. Proxy unlocks the escrowed collateral and returns it to the trader.

4. Message lifecycle and state machine

Every cross-chain intent moves through a three-state lifecycle:





4.1 Idempotency

Each message carries a **unique `messageId`** returned by `sendMessage()`. The receiving contract (home market or proxy) **MUST** deduplicate messages:

```

mapping(bytes32 => bool) public processedMessages;

function receiveMessage(
    uint64 sourceChainSelector,
    address sender,
    bytes calldata payload
) external override {
    bytes32 messageId = keccak256(abi.encodePacked(
        sourceChainSelector, sender, payload
    ));

    require(!processedMessages[messageId], "Already processed");
    processedMessages[messageId] = true;

    // Decode and execute...
}
  
```

Why: Bridge messaging is **at-least-once delivery**. The same message may arrive multiple times (reorg, re-transmission). On-chain idempotency makes at-least-once effectively-once.

4.2 Ordering and nonces

Each intent includes a **per-trader nonce**. The home market enforces monotonically increasing nonces per `(trader, proxyChain)` pair:

```

mapping(address => mapping(uint64 => uint256)) public traderNonce;

function _validateIntent(BuyIntent memory intent, uint64 sourceChain) internal {
    require(
        intent.nonce == traderNonce[intent.trader][sourceChain] + 1,
        "Nonce out of order"
    );
    traderNonce[intent.trader][sourceChain] = intent.nonce;
}
  
```

Why: Prevents replay attacks and guarantees in-order execution when multiple intents from the same trader are in flight.

5. Guarantees and non-guarantees

Property	Guarantee
Delivery	At-least-once. The bridge adapter retries until delivery confirms or the message expires (adapter-specific timeout). On-chain idempotency deduplicates retries → effectively-once execution.
Ordering	Per-trader ordering via nonces; no global ordering across traders. If trader Alice sends intent A then B, home market processes A before B. Intents from Alice and Bob may execute in any interleaving.
Finality	Asynchronous. A bridged buy confirms in bridge time (seconds to minutes), not block time. The UI quotes a <i>guaranteed-bounds fill envelope</i> at intent time; the home AMM fills within it or refunds.
Atomicity	None. A bridged trade is two transactions (proxy lock → home fill → proxy mint). If the home fill reverts, the RefundNotice unlocks the proxy collateral — the trader's net position is unchanged.
Price guarantees	Bounded fill or refund. The proxy quotes a price range (current spot ± maxSlippage); the home AMM fills within that range or issues a RefundNotice. The trader never receives worse than quoted.
Censorship resistance	Depends on the bridge adapter. CCIP and LayerZero use decentralized oracle networks; a malicious adapter can censor. Adapter allowlists + per-route exposure caps mitigate (see doc 06).

The user-facing promise: "Your buy will execute at the quoted price (± max slippage) or your collateral is refunded. Confirmation takes bridge time, not block time."

6. The Chainlink CCIP tie — explicit and foundational

Justify's `IBridgeAdapter` is **directly inspired by Chainlink CCIP's messaging architecture**:

- **Chain selectors:** CCIP uses 64-bit `uint64` chain identifiers instead of raw chain IDs, decoupling the protocol from EVM chain-ID namespace collisions. Justify adopts the same `destChainSelector` type.
- **Router/Client model:** CCIP separates the sender's call (`IRouterClient.ccipSend`) from the receiver's callback (`IAny2EVMMessageReceiver.ccipReceive`). Justify mirrors this with `sendMessage` (router-style) and `receiveMessage` (callback).
- **Typed message structs:** CCIP's `Client.EVM2AnyMessage` is a struct with destination, receiver, data, and token instructions. Justify's `BuyIntent`, `SellIntent`, etc., are typed payloads following the same pattern.
- **Message ID returns:** Both CCIP and Justify return `bytes32 messageId` for tracking, idempotency, and linking confirmations to intents.

The CCIP adapter (document 02) is the reference implementation of `IBridgeAdapter`. When Justify bridges Arc ↔ Base, the adapter wraps `IRouterClient.ccipSend` and implements `IAny2EVMMessageReceiver.ccipReceive` to forward messages into Justify's `receiveMessage`. The interface is abstract enough to support other bridges (LayerZero, Wormhole), but CCIP's design is the blueprint.

7. Forward links

- [02-chainlink-ccip-adapter.md](#) — The concrete CCIP adapter implementing this interface; the explicit mapping from Justify's message types to CCIP's `EVM2AnyMessage` and CCTP for USDC legs.
 - [03-marketproxy-and-home-market.md](#) — How `MarketProxy` calls `IBridgeAdapter.sendMessage()` on buy/sell and implements `receiveMessage()` to process confirmations.
 - [06-security-and-risk-management.md](#) — Adapter allowlists, per-route exposure caps, and the circuit breaker (analogous to CCIP's Risk Management Network).
-

Summary: `IBridgeAdapter` is the **transport seam** in Justify's bridge architecture. It is an injected dependency (parameter, not commitment), inspired by Chainlink CCIP's Router/Client model, carrying typed message payloads with guaranteed-bounds

fills or refunds, and enforcing idempotency + ordering via message IDs and per-trader nonces. One interface, many transports
— the bridge is configuration, not architecture.

Chainlink CCIP Adapter — Justify's First Production Bridge Transport

Document: 02-chainlink-ccip-adapter.md

Status: Design (Phase 3 — Bridge-proxied markets)

Relates to: Whitepaper §5 (bridge-proxied markets), §5.3 (resolution phases)

Dependencies: [01-bridge-adapter-interface.md](#)

1. Explicit acknowledgment

Justify's entire bridge layer architecture is inspired by Chainlink CCIP's design.

The first production bridge transport for Justify is **Chainlink CCIP** (Cross-Chain Interoperability Protocol), and the architectural decomposition of Justify's own bridge contracts — the separation of a chain-local entry point from per-route adapters, the abstraction of message transport, the independent risk management layer, the commitment-then-execution pattern — is **directly modeled on Chainlink's CCIP architecture**.

We credit this inspiration explicitly:

- Justify's `IBridgeAdapter` interface is modeled on CCIP's router/transport separation.
- Justify's per-route exposure caps and circuit-breaker pattern mirror CCIP's **Risk Management Network (RMN)**.
- Justify's message commitment model (intent submitted, bridge confirms, outcome minted/refunded) is structurally analogous to CCIP's commit/execute DON phases.
- Justify's first concrete adapter, `CcipBridgeAdapter`, directly calls Chainlink's own `IRouterClient` contracts.

This document describes the CCIP architecture primer, the mapping from Justify concepts to CCIP concepts, and the Solidity design of the `CcipBridgeAdapter` that implements `IBridgeAdapter` on top of CCIP.

2. Chainlink CCIP architecture primer

Chainlink CCIP is a **general-purpose cross-chain messaging protocol** designed to enable arbitrary message and token transfers between EVM and non-EVM blockchains with cryptographic security guarantees, decentralized verification, and an independent risk management layer.

2.1 Core components

2.1.1 Router contract (per chain)

The **Router** (`IRouterClient`) is the single entry point on each supported chain for sending CCIP messages.

- Every CCIP sender calls `router.ccipSend(destinationChainSelector, message)` to initiate a cross-chain message.
- The router validates the message, calculates fees (in LINK or native gas token), collects payment, and forwards the message to the appropriate **OnRamp** contract for the destination lane.

2.1.2 OnRamp and OffRamp contracts (per lane)

A **lane** is a directed path from one source chain to one destination chain (e.g., Ethereum → Base, or Base → Polygon). Each lane has:

- **OnRamp (source chain):** receives validated messages from the Router, serializes them into a canonical format, emits a `CCIPSendRequested` event for off-chain observation, and increments a sequence number. The OnRamp is the source-chain truth for "this message was requested."
- **OffRamp (destination chain):** receives executed messages from the Executing DON, verifies inclusion in the committed Merkle root, executes the message (calls `ccipReceive` on the target contract), and emits a `CCIPMessageExecuted` event. The OffRamp is the destination-chain truth for "this message was delivered."

2.1.3 Commit and Executing DONs (Decentralized Oracle Networks)

CCIP's off-chain infrastructure consists of two independent oracle networks:

- **Committing DON:** observes OnRamp events on the source chain, constructs a Merkle root of all pending messages for a given lane batch, and submits that root to the destination OffRamp via a `commit()` transaction. This establishes cryptographic finality: the destination chain knows a message exists without yet executing it.
- **Executing DON:** observes committed roots, constructs Merkle proofs for individual messages, calls the OffRamp's `execute()` with the proof, which verifies inclusion and delivers the message to the recipient contract's `ccipReceive()` function.

The two-phase design (commit, then execute) ensures:

1. **Cryptographic finality** before execution — no reliance on single-oracle honesty.
2. **Gas efficiency** — one commit covers a batch of messages; execution happens per message only when needed.

2.1.4 Risk Management Network (RMN)

The **RMN** is an **independent** set of nodes (distinct from the DONs) that continuously monitors CCIP lanes for anomalies:

- **Blessing:** the RMN observes committed Merkle roots and, if valid, "blesses" them — a precondition for execution.
- **Cursing:** if the RMN detects an invalid root or anomalous behavior (e.g., a compromised DON), it "curses" the lane, immediately halting all executions on that route until governance intervention.

The RMN is CCIP's defense-in-depth layer: even if both DONs are compromised, the RMN can prevent invalid state from finalizing on the destination chain.

2.1.5 Chain selectors

CCIP uses **chain selectors** — uint64 identifiers distinct from EVM chain IDs — to name blockchains in a multi-ecosystem namespace. Examples:

Chain	EVM Chain ID	CCIP Chain Selector
Ethereum	1	5009297550715157269
Base	8453	15971525489660198786
Polygon PoS	137	4051577828743386545
BSC	56	11344663589394136015

When Justify's adapter sends a message, it translates the destination chain's EVM chain ID to its CCIP chain selector via a registry mapping.

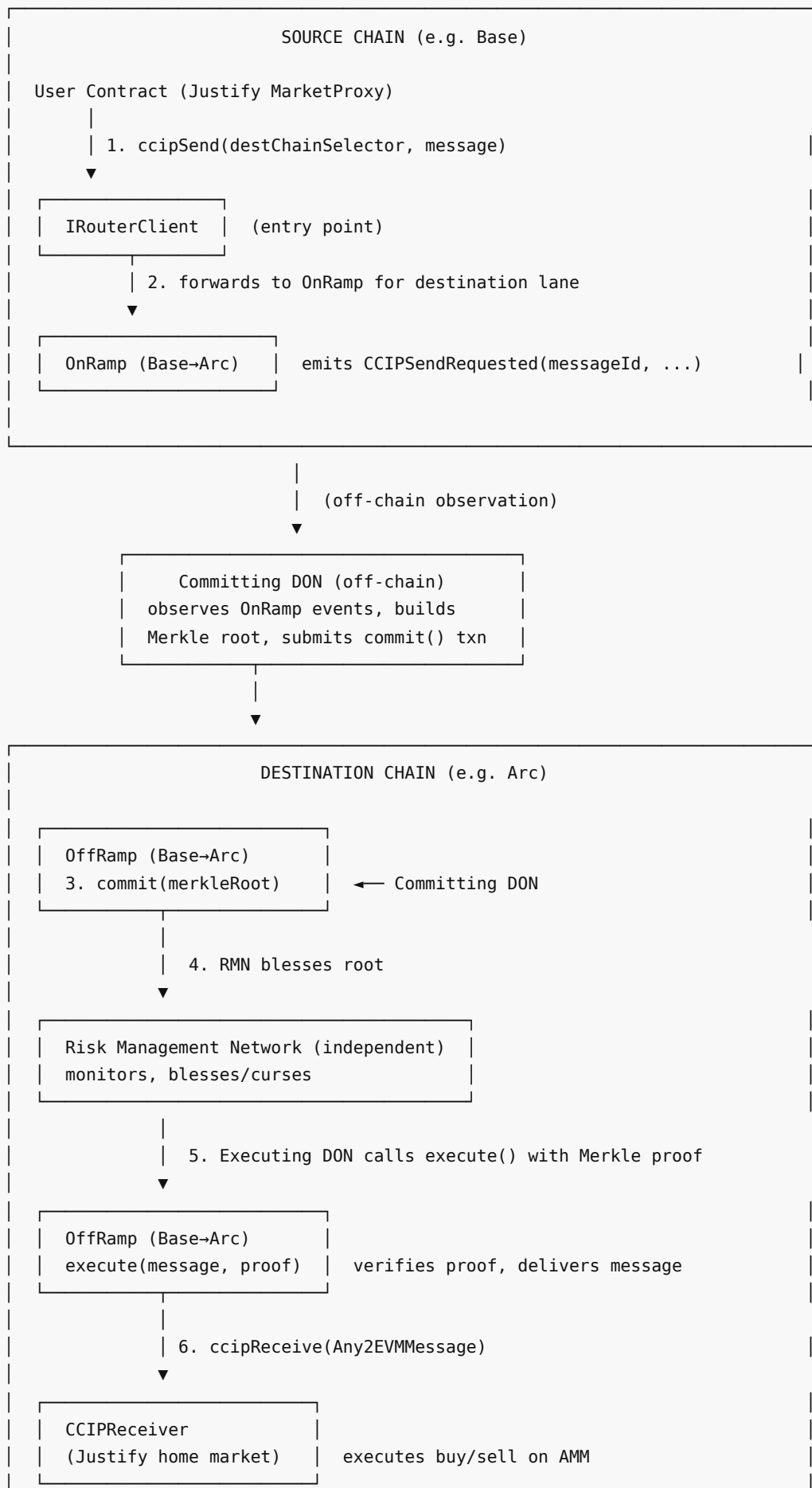
2.1.6 Token transfer model

CCIP supports **native token transfers** alongside arbitrary messages via the **Token Pool** model:

- Each token (e.g., USDC, LINK) has a **token pool contract** per chain that locks/mints or burns/unlocks tokens on send/receive.
- Senders specify token amounts in the `Client.EVM2AnyMessage.tokenAmounts` array; CCIP coordinates the token leg in parallel with the data leg.
- For USDC specifically, Chainlink integrates with **Circle CCTP** (Cross-Chain Transfer Protocol) on supported lanes, enabling native USDC burns/mints rather than locked/wrapped representations.

Justify uses CCIP for **message-only transfers** (buy/sell intents), and **separately** uses Circle CCTP for the value leg (USDC collateral transfers) when the lane supports it (see [04-per-chain-bridge-playbook.md](#)).

2.2 CCIP message flow (ASCII diagram)



Key properties:

- **Asynchronous:** steps 3–6 occur seconds to minutes after step 1, depending on source/destination finality and DON cadence.
- **Cryptographic finality:** the Merkle root commitment (step 3) is immutable; execution (step 6) is proof-based.
- **Defense in depth:** both DONs and the RMN must agree; any single layer can halt invalid state.

2.3 Programming model: Client.EVM2AnyMessage and ccipReceive

Sending a message

```
import {IRouterClient} from "@chainlink/contracts-ccip/src/v0.8/ccip/interfaces/IRouterClient.sol";
import {Client} from "@chainlink/contracts-ccip/src/v0.8/ccip/libraries/Client.sol";

IRouterClient router = IRouterClient(ROUTER_ADDRESS);

Client.EVM2AnyMessage memory message = Client.EVM2AnyMessage({
    receiver: abi.encode(destinationContractAddress), // ABI-encoded destination
    data: abi.encode(customPayload),                // arbitrary bytes
    tokenAmounts: new Client.EVMTokenAmount[](0),    // empty for data-only
    extraArgs: Client._argsToBytes(
        Client.EVMExtraArgsV1({gasLimit: 200_000}) // execution gas on destination
    ),
    feeToken: address(linkToken)                     // pay fee in LINK
});

uint256 fee = router.getFee(destinationChainSelector, message);
linkToken.approve(address(router), fee);

bytes32 messageId = router.ccipSend(destinationChainSelector, message);
// messageId is unique, can be indexed for tracking
```

Receiving a message

```
import {CCIPReceiver} from "@chainlink/contracts-ccip/src/v0.8/ccip/applications/CCIPReceiver.sol";
import {Client} from "@chainlink/contracts-ccip/src/v0.8/ccip/libraries/Client.sol";

contract MyReceiver is CCIPReceiver {
    constructor(address router) CCIPReceiver(router) {}

    function _ccipReceive(Client.Any2EVMMessage memory message) internal override {
        address sender = abi.decode(message.sender, (address));
        bytes memory payload = message.data;
        uint64 sourceChainSelector = message.sourceChainSelector;

        // validate sender, parse payload, execute business logic
    }
}
```

The `_ccipReceive` function is called by the OffRamp when a message is executed. The receiver **must** inherit `CCIPReceiver` and override `_ccipReceive`.

3. CcipBridgeAdapter design

Justify's `CcipBridgeAdapter` is a concrete implementation of the `IBridgeAdapter` interface (defined in [01-bridge-adapter-interface.md](#)) that uses Chainlink CCIP as the transport layer.

3.1 Responsibilities

The adapter must:

1. **Send cross-chain intents:** when a user submits a buy/sell order on a `MarketProxy`, the adapter calls `IRouterClient.ccipSend()` to send the intent (question ID, outcome, amount, user address) to the home market on the destination chain.
2. **Receive and route incoming messages:** when the home market confirms a fill or resolution, the adapter receives the CCIP message via `ccipReceive()` and routes it to the appropriate `MarketProxy` contract for local state update (mint wrapped position receipt, or settle redemption).
3. **Map Justify routes to CCIP lanes:** maintain a registry of (Justify destination chain ID) → (CCIP chain selector) and (route) → (allowed sender/receiver addresses).
4. **Pay CCIP fees:** calculate and collect CCIP transport fees from the user or protocol treasury, payable in LINK or native gas token.
5. **Emit adapter events:** for off-chain indexing, emit `IntentSent`, `MessageReceived`, `IntentFailed` events that mirror the CCIP message lifecycle into Justify's event schema.

3.2 Contract structure

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import {IBridgeAdapter} from "../IBridgeAdapter.sol";
import {IRouterClient} from "@chainlink/contracts-ccip/src/v0.8/ccip/interfaces/IRouterClient.sol";
import {CCIPReceiver} from "@chainlink/contracts-ccip/src/v0.8/ccip/applications/CCIPReceiver.sol";
import {Client} from "@chainlink/contracts-ccip/src/v0.8/ccip/libraries/Client.sol";
import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

/// @title CcipBridgeAdapter
/// @notice Chainlink CCIP adapter for Justify cross-chain market intents
/// @dev Implements IBridgeAdapter; sends via IRouterClient, receives via CCIPReceiver
contract CcipBridgeAdapter is IBridgeAdapter, CCIPReceiver, Ownable {
    // _____
    // State
    // _____

    /// @notice LINK token for fee payment (optional; can also pay in native)
    IERC20 public immutable linkToken;

    /// @notice Whether to pay fees in LINK (true) or native gas token (false)
    bool public payFeesInLink;

    /// @notice Mapping: Justify destination chain ID → CCIP chain selector
    mapping(uint256 => uint64) public chainIdToSelector;

    /// @notice Mapping: CCIP chain selector → Justify chain ID (reverse lookup)
    mapping(uint64 => uint256) public selectorToChainId;
```

```

    /// @notice Allowlisted remote adapter addresses per CCIP chain selector
    /// Only messages from these addresses are accepted
    mapping(uint64 => address) public allowedSenders;

    /// @notice Target contract address on each destination chain (typically the home market or
    MarketProxy registry)
    mapping(uint64 => address) public destinationTargets;

    /// @notice Gas limit for ccipReceive execution on destination chain
    uint256 public destinationGasLimit = 200_000;

    // _____
    // Events
    // _____

    event IntentSent(
        bytes32 indexed ccipMessageId,
        uint256 indexed destinationChainId,
        address indexed user,
        bytes intentPayload,
        uint256 fee
    );

    event MessageReceived(
        bytes32 indexed ccipMessageId,
        uint64 indexed sourceChainSelector,
        address indexed sender,
        bytes payload
    );

    event RouteConfigured(
        uint256 indexed justifyChainId,
        uint64 indexed ccipSelector,
        address allowedSender,
        address destinationTarget
    );

    // _____
    // Constructor
    // _____

    constructor(
        address _router,
        address _linkToken,
        bool _payFeesInLink
    ) CCIPReceiver(_router) Ownable(msg.sender) {
        linkToken = IERC20(_linkToken);
        payFeesInLink = _payFeesInLink;
    }

    // _____
    // IBridgeAdapter implementation
    // _____

    /// @notice Send a cross-chain intent (buy/sell) via CCIP

```

```

/// @param destinationChainId Justify chain ID (translated to CCIP selector internally)
/// @param payload ABI-encoded intent data (market ID, outcome, amount, user, etc.)
/// @return messageId Unique CCIP message identifier for tracking
function sendIntent(
    uint256 destinationChainId,
    bytes calldata payload
) external payable override returns (bytes32 messageId) {
    uint64 destSelector = chainIdToSelector[destinationChainId];
    require(destSelector != 0, "CcipAdapter: destination not configured");

    address target = destinationTargets[destSelector];
    require(target != address(0), "CcipAdapter: no target for destination");

    // Build CCIP message
    Client.EVM2AnyMessage memory message = Client.EVM2AnyMessage({
        receiver: abi.encode(target),
        data: payload,
        tokenAmounts: new Client.EVMTokenAmount[](0), // data-only, no token transfer
        extraArgs: Client._argsToBytes(
            Client.EVMExtraArgsV1({gasLimit: destinationGasLimit})
        ),
        feeToken: payFeesInLink ? address(linkToken) : address(0)
    });

    // Calculate fee
    IRouterClient router = IRouterClient(i_ccipRouter);
    uint256 fee = router.getFee(destSelector, message);

    // Pay fee
    if (payFeesInLink) {
        require(linkToken.balanceOf(msg.sender) >= fee, "CcipAdapter: insufficient LINK");
        linkToken.transferFrom(msg.sender, address(this), fee);
        linkToken.approve(address(router), fee);
    } else {
        require(msg.value >= fee, "CcipAdapter: insufficient native fee");
    }

    // Send via CCIP
    messageId = router.ccipSend{value: payFeesInLink ? 0 : fee}(destSelector, message);

    emit IntentSent(messageId, destinationChainId, msg.sender, payload, fee);
}

/// @notice Query the fee for sending an intent to a destination chain
function getFee(
    uint256 destinationChainId,
    bytes calldata payload
) external view override returns (uint256) {
    uint64 destSelector = chainIdToSelector[destinationChainId];
    require(destSelector != 0, "CcipAdapter: destination not configured");

    address target = destinationTargets[destSelector];
    require(target != address(0), "CcipAdapter: no target for destination");

    Client.EVM2AnyMessage memory message = Client.EVM2AnyMessage({
        receiver: abi.encode(target),

```

```

        data: payload,
        tokenAmounts: new Client.EVMTokenAmount[](0),
        extraArgs: Client._argsToBytes(
            Client.EVMExtraArgsV1({gasLimit: destinationGasLimit})
        ),
        feeToken: payFeesInLink ? address(linkToken) : address(0)
    });

    IRouterClient router = IRouterClient(i_ccipRouter);
    return router.getFee(destSelector, message);
}

// _____
// CCIPReceiver implementation
// _____

/// @notice Receive and route a CCIP message from another chain
/// @dev Called by the CCIP OffRamp; validates sender, decodes payload, forwards to local
handler
function _ccipReceive(
    Client.Any2EVMMMessage memory message
) internal override {
    uint64 sourceSelector = message.sourceChainSelector;
    address sender = abi.decode(message.sender, (address));

    // Validate sender is allowlisted for this source chain
    require(
        allowedSenders[sourceSelector] == sender,
        "CcipAdapter: sender not allowed"
    );

    bytes memory payload = message.data;
    bytes32 messageId = message.messageId;

    emit MessageReceived(messageId, sourceSelector, sender, payload);

    // Decode and route payload to appropriate handler (e.g., MarketProxy or home market)
    // Implementation depends on the message type encoded in payload
    // Example: (uint8 msgType, bytes data) = abi.decode(payload, (uint8, bytes));
    // if (msgType == MSG_TYPE_FILL_CONFIRMATION) { ... }
    // This routing logic is specific to Justify's intent schema and is defined in the
    // broader bridge integration (see 03-marketproxy-and-home-market.md)

    _routeReceivedMessage(sourceSelector, payload);
}

/// @notice Internal routing of received messages (stub; implementation per message type)
function _routeReceivedMessage(uint64 sourceSelector, bytes memory payload) internal {
    // Decode message type and forward to appropriate contract
    // E.g., call MarketProxy.confirmFill(...) or MarketProxy.settleResolution(...)
    // Concrete implementation depends on Justify message schema
}

// _____
// Admin functions
// _____

```

```

/// @notice Configure a new route: map Justify chain ID ↔ CCIP selector, allowlist sender
function configureRoute(
    uint256 justifyChainId,
    uint64 ccipSelector,
    address allowedSender,
    address destinationTarget
) external onlyOwner {
    chainIdToSelector[justifyChainId] = ccipSelector;
    selectorToChainId[ccipSelector] = justifyChainId;
    allowedSenders[ccipSelector] = allowedSender;
    destinationTargets[ccipSelector] = destinationTarget;

    emit RouteConfigured(justifyChainId, ccipSelector, allowedSender, destinationTarget);
}

/// @notice Update destination gas limit for ccipReceive execution
function setDestinationGasLimit(uint256 _gasLimit) external onlyOwner {
    destinationGasLimit = _gasLimit;
}

/// @notice Switch between LINK and native fee payment
function setPayFeesInLink(bool _payInLink) external onlyOwner {
    payFeesInLink = _payInLink;
}

/// @notice Withdraw stuck LINK or native tokens (safety escape hatch)
function withdraw(address token, uint256 amount) external onlyOwner {
    if (token == address(0)) {
        payable(owner()).transfer(amount);
    } else {
        IERC20(token).transfer(owner(), amount);
    }
}
}

```

3.3 Key design choices

1. **Chain ID translation:** Justify uses standard EVM chain IDs (1, 8453, 137, etc.); CCIP uses uint64 chain selectors. The adapter maintains a bidirectional mapping configured per route by the protocol owner.
2. **Sender allowlisting:** only messages from pre-configured remote adapter addresses (e.g., the `CcipBridgeAdapter` instance on the source chain) are accepted. This prevents spoofing.
3. **Fee flexibility:** CCIP fees can be paid in LINK (preferred for rate stability) or native gas token (simpler UX). The adapter supports both; the protocol chooses per deployment.
4. **Gas limit:** the destination execution gas is set per adapter deployment (default 200k gas for a typical fill confirmation or resolution settlement). Adjustable by owner if market logic complexity grows.
5. **Message routing stub:** the `_routeReceivedMessage` function is a stub in this base adapter. Concrete routing (e.g., "if this is a fill confirmation, call `MarketProxy.confirmFill()`") is implemented in an integration layer or extended adapter that knows about Justify's market registry. This keeps the adapter focused on CCIP transport.

4. Justify ↔ Chainlink CCIP concept mapping

Justify concept	Chainlink CCIP equivalent	Notes
Bridge entrypoint	IRouterClient (Router contract)	Justify's MarketProxy calls <code>CcipBridgeAdapter.sendIntent()</code> , which calls CCIP's <code>router.ccipSend()</code> .
Route (source chain → dest chain)	Lane (source chain → dest chain)	Each Justify route (e.g., Base → Arc) maps to one CCIP lane with its own OnRamp/OffRamp.
Adapter send	OnRamp + ccipSend	<code>CcipBridgeAdapter.sendIntent()</code> → <code>router.ccipSend()</code> → OnRamp emits <code>CCIPSendRequested</code> .
Adapter receive	OffRamp + ccipReceive	CCIP OffRamp calls <code>CcipBridgeAdapter._ccipReceive()</code> to deliver the message.
Message ID	CCIP messageId (bytes32)	Unique identifier returned by <code>ccipSend()</code> , indexed for tracking and event correlation.
Destination chain ID	CCIP chain selector (uint64)	Justify translates EVM chain IDs to CCIP selectors via <code>chainIdToSelector</code> registry.
Exposure cap / circuit breaker	Risk Management Network (RMN) — blessing/cursing	Justify's circuit breaker (see 06-security-and-risk-management.md) mirrors RMN's role: halt on anomaly detection.
Intent payload	Client.EVM2AnyMessage.data (arbitrary bytes)	Justify encodes (marketId, outcome, amount, user) into <code>message.data</code> ; CCIP treats it as opaque.
Fee model	CCIP fee in LINK or native	Calculated by <code>router.getFee()</code> , paid upfront per message; Justify surfaces this in the UI payout quote.
Sender authentication	Allowlisted sender addresses per CCIP receiver	Justify's <code>allowedSenders</code> mapping enforces that only authorized remote adapters can deliver messages.
Resolution broadcast	Same CCIP send/receive (home → all proxies)	When the home market resolves, the resolver sends resolution messages via CCIP to all proxy chains (see 05-cross-chain-resolution.md).
Two-phase execution (lock → mint)	Commit DON → Executing DON (commit root → execute proof)	Justify's intent lifecycle (submitted → confirmed → minted) mirrors CCIP's commit/execute pattern, though at a higher semantic level.

5. Token transfers vs message-only: when to use CCIP vs CCTP

Chainlink CCIP supports **both** arbitrary messages and native token transfers (via Token Pools). However, for Justify's use case, we employ a **hybrid strategy**:

5.1 Message leg: CCIP

What: Buy/sell intents, fill confirmations, resolution settlements — all messages that carry **instructions, not value**.

How: CCIP arbitrary messaging (the `data` field of `Client.EVM2AnyMessage`), as implemented in `CcipBridgeAdapter`.

Why:

- CCIP's decentralized DON model provides cryptographic finality and RMN-backed security for the instruction layer.
- The message is small (< 1 KB), so CCIP's fee is proportional and predictable.
- CCIP natively supports non-EVM destinations (future-proofing) and handles retries/failures gracefully.

5.2 Value leg: Circle CCTP (where available)

What: The actual USDC collateral transfer from the user's source chain to the home market's pool (or back on redemption).

How: Circle CCTP (Cross-Chain Transfer Protocol) — a specialized USDC bridge where USDC is burned on the source chain and natively minted on the destination chain via Circle's attestation service.

Why:

- **Native USDC:** CCTP mints canonical USDC on the destination, not a wrapped or pool-backed synthetic. This is critical for Justify's collateral accounting, which assumes chain-native USDC.
- **Lower cost:** CCTP fees are lower than general-purpose CCIP token transfers for USDC specifically.
- **Circle's security model:** CCTP is Circle's own infrastructure, aligned with USDC's trust model. For a stablecoin-denominated protocol, this is the natural choice.
- **Lane coverage:** CCTP supports Ethereum ↔ Base, Ethereum ↔ Polygon, Base ↔ Polygon, and many other high-volume routes — the exact lanes Justify targets.

Where CCTP is not available: for lanes where CCTP does not yet support both endpoints (e.g., BSC ↔ Arc, hypothetically), Justify can fall back to CCIP token transfers via a USDC token pool, or employ a lock-on-source / mint-wrapped model. The adapter design is **transport-agnostic at the IBridgeAdapter seam**; the per-chain playbook (see [04-per-chain-bridge-playbook.md](#)) documents which transport is used per lane.

5.3 Separation of concerns

```
User submits buy intent on MarketProxy (source chain)
|
├─ USDC collateral: burned via CCTP, minted on home chain into AMM pool
|
└─ Intent message: sent via CCIP to home market, triggers AMM.buy()
    |
    └─ Fill confirmation: CCIP message back to source MarketProxy
        |
        └─ Wrapped receipt minted to user's address
```

The two legs are **decoupled**: CCIP handles "what to do" (the trade instruction), CCTP handles "where the money goes" (the value settlement). This mirrors how traditional finance separates instruction settlement (FIX protocol) from cash settlement (wire transfer).

6. Honest limits and operational considerations

6.1 CCIP lane availability

CCIP does not support every chain pair yet. As of this document's writing (2026-06), CCIP lanes exist for:

- **Ethereum ↔ Base** ✓
- **Ethereum ↔ Polygon** ✓
- **Base ↔ Polygon** ✓
- **Ethereum ↔ BSC** ✓
- **Arc ↔ {Ethereum, Base, Polygon, BSC}** — depends on Arc mainnet CCIP integration roadmap (testnet may require Chainlink pilot deployment)

Implication for Justify: Phase 3 rollout must sequence per lane availability. If Arc ↔ Base is the first target (per the whitepaper), Chainlink's Arc testnet CCIP deployment is a **critical path dependency**. Fallback: use a generalized message bridge (e.g., LayerZero, Wormhole) for lanes where CCIP is not yet live, with a migration plan once CCIP coverage expands (see [01-bridge-adapter-interface.md](#) for multi-adapter support).

6.2 Finality-dependent latency

CCIP's commit phase waits for **source-chain finality** before committing the Merkle root to the destination. Finality times:

Chain	Finality mechanism	Typical time
Ethereum	LMD-GHOST (2 epochs)	~15 minutes
Base	OP Stack (L1 finality)	~15 minutes
Polygon PoS	Heimdall checkpoints	~30 minutes
BSC	Parlia (21 validators)	~1 minute
Arc	(depends on Arc consensus)	TBD (~1 minute)

Implication: A buy intent sent from Ethereum to Base will take ~15 minutes to reach the Commit DON, then another ~1–2 minutes for execution. The Justify UI must surface this as **"Your order is being bridged — expected fill in ~15–20 minutes"** rather than pretending it is instant. The `MarketProxy` design includes a **guaranteed-bounds quote** (max slippage) at intent submission time, so the user knows their worst-case fill price upfront (see [03-marketproxy-and-home-market.md](#)).

6.3 Fee in LINK or native

CCIP fees are paid in LINK (Chainlink's native token) or the chain's native gas token (ETH, POL, BNB, etc.). Fee rate depends on:

- Lane congestion (OnRamp/OffRamp utilization)
- Destination gas limit (higher execution complexity → higher fee)
- Current LINK or native token price

Implication: The `CcipBridgeAdapter` exposes `getFee()` for upfront quotes. The Justify UI calls this **before** the user confirms the order, and displays the bridge fee explicitly in the payout calculator ("To win: \$X, after \$Y trade fee and \$Z bridge fee"). For LINK-denominated fees, the protocol must maintain a LINK balance on each source chain or enable users to pay in native and auto-swap to LINK (via Chainlink Automation or a DEX integration).

6.4 Allowlisting senders and receivers

CCIP best practice: **allowlist** which contracts can send to your receiver, and which receivers your sender is authorized to target. Justify enforces this via:

- `CcipBridgeAdapter.allowedSenders[ccipSelector] = remoteAdapterAddress` — only accept messages from known remote adapter instances.
- `CcipBridgeAdapter.destinationTargets[ccipSelector] = homeMarketOrProxyRegistry` — only send to known endpoints.

Implication: When deploying a new chain, the protocol owner must call `configureRoute()` on both sides (source and destination adapters) to establish the two-way lane. This is an operational checklist item (see [04-per-chain-bridge-playbook.md](#)).

6.5 Not yet deployed

This entire design is forward-looking. As of Phase 1 (Arc testnet), Justify is single-chain. Phase 2 (multi-chain native) will deploy independent markets per chain with no bridge. Phase 3 (bridge-proxied markets) is when `CcipBridgeAdapter` is built, tested, and deployed.

Why document it now? To freeze the `IBridgeAdapter` interface and the architectural separation before the contracts are written, so that the `MarketProxy` and home-market integration can be designed in parallel without waiting for CCIP lane availability.

7. Forward links and integration

- [01-bridge-adapter-interface.md](#) — the `IBridgeAdapter` seam this adapter implements.

- [03-marketproxy-and-home-market.md](#) — how `MarketProxy` calls `sendIntent()` and handles `_ccipReceive()` callbacks.
 - [04-per-chain-bridge-playbook.md](#) — operational playbook: how to deploy the adapter per chain, configure lanes, fund LINK, test end-to-end.
 - [05-cross-chain-resolution.md](#) — how CCIP carries resolution messages from the home market to all proxies, settling every chain in one atomic broadcast.
 - [06-security-and-risk-management.md](#) — circuit breaker, exposure caps, RMN monitoring, and what happens when CCIP itself is compromised.
-

8. Summary

Justify's bridge layer is **explicitly inspired by Chainlink CCIP's architecture** — the router/adapter separation, the commit/execute DON model, the independent risk management network, and the lane-based transport abstraction. Our first production adapter, `CcipBridgeAdapter`, is not a reimplement of CCIP but a **thin integration layer** that calls CCIP's own `IRouterClient` to send intents and implements `CCIPReceiver` to receive confirmations.

This design gives Justify:

- **Decentralized security:** CCIP's DON consensus and RMN blessing/cursing protect the message layer.
- **Predictable latency:** finality-bound commit phase means users know when to expect their fill.
- **Cost transparency:** upfront fee quotes let the UI surface the full cost before submission.
- **Separation of value and instruction:** CCIP for messages, CCTP for USDC transfers, each optimized for its role.
- **Future composability:** the same adapter pattern can wrap LayerZero, Wormhole, or any other message bridge if a lane requires it.

The result: Justify's cross-chain markets are **bridge-transport-agnostic by contract design**, but **CCIP-first by deployment strategy**, leveraging the most mature and security-hardened cross-chain protocol in production today.

Next: Read [03-marketproxy-and-home-market.md](#) to see how `MarketProxy` wraps this adapter into a user-facing order flow.

03 — MarketProxy and Home-Market Wiring

Lock, Bridge, Mint: How remote-chain users trade home-chain markets

This document describes the `MarketProxy` contract design (one per remote chain per market) and the home-side `HomeMarketGateway` integration that together implement the **home/proxy split** from [whitepaper §5.1–§5.2](#). The architecture is directly inspired by **Chainlink CCIP's sender/Router → OffRamp/receiver separation**, where the proxy plays the sender role and the home gateway plays the receiver role, with `IBridgeAdapter` ([doc 01](#)) as the transport seam.

1. The model recap

User on Base (wants to buy YES on a market homed on Arc)
↓ buy(YES, 100 USDC)

```
MarketProxy (Base, for Arc market #42)
1. Lock 100 USDC in escrow
2. Emit BuyIntent via IBridgeAdapter
3. Wait for FillConfirmation
4. Mint wrapped receipt (ERC-20) to user
```

↓ bridge message (CCIP, CCTP)

```
HomeMarketGateway (Arc)
1. Receive BuyIntent
2. Execute buy against MarketAMM #42
3. Hold outcome tokens on behalf of proxy
4. Send FillConfirmation back to proxy
```

Key properties (from whitepaper §5.2 table):

Property	Implementation
Unified liquidity	One AMM pool on the home chain; all proxies route to it → single source of truth for pricing.
Single source of truth	Home-market state (Open/Closed/Resolved) propagates to proxies; no proxy can disagree.
Asynchronous fills	Bridge time (seconds to minutes); guaranteed-bounds fill or refund.
Bridge trust	Proxy position is as safe as min(home chain security, bridge adapter security).
Local-first routing	If user's chain == market's home chain, bypass proxy entirely → native PredictionMarket buy.

2. MarketProxy contract design

2.1 Purpose

A **MarketProxy** is deployed **on a remote chain** (e.g. Base, Polygon, BSC) for a specific **home market** (e.g. Arc market #42). It:

- Locks local collateral (USDC) in escrow when a user submits a buy/sell.
- Calls `IBridgeAdapter.sendMessage()` to emit a `BuyIntent` or `SellIntent` to the home chain.
- Mints a **wrapped position receipt** (ERC-20 representing outcome tokens held on the home chain) when the fill confirmation arrives.
- Handles refunds if the home AMM cannot fill within the user's slippage bounds.

- Manages redemption after market resolution: burn receipt → release local collateral.

2.2 Illustrative Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import "./IBridgeAdapter.sol";

/// @title MarketProxy
/// @notice Remote-chain proxy for a home-chain prediction market.
///         Locks local collateral, emits cross-chain buy/sell intents to the
///         home market, mints wrapped position receipts, and handles redemption
///         after settlement propagates back.
///
///         Architecture inspired by Chainlink CCIP sender → OffRamp/receiver:
///         this proxy is the sender; HomeMarketGateway on the home chain is
///         the receiver; IBridgeAdapter is the transport seam.
contract MarketProxy is ReentrancyGuard {
    using SafeERC20 for IERC20;

    /// @notice The home chain where the real PredictionMarket + AMM lives.
    uint256 public immutable homeChainId;

    /// @notice The market ID on the home chain this proxy fronts.
    uint256 public immutable homeMarketId;

    /// @notice Local collateral token (USDC on Base, Polygon, etc.).
    IERC20 public immutable collateral;

    /// @notice The bridge adapter (CCIP, LayerZero, etc.) used to send messages.
    IBridgeAdapter public immutable bridgeAdapter;

    /// @notice HomeMarketGateway address on the home chain.
    address public immutable homeGateway;

    /// @notice Wrapped position receipt tokens (ERC-20) per outcome.
    ///         wrappedOutcome[0] = wrapped YES, wrappedOutcome[1] = wrapped NO.
    WrappedOutcomeToken[2] public wrappedOutcome;

    /// @notice Intent nonce (prevents replay / double-fill).
    uint256 private _nextIntentNonce;

    /// @notice Pending intent state: collateral locked, awaiting fill or refund.
    struct PendingIntent {
        address user;
        uint8 outcomeIndex;
        uint256 collateralLocked;
        uint256 minSharesOut;
        uint256 timestamp;
    }
    mapping(uint256 => PendingIntent) public pendingIntents;
```

```

/// @notice Market state mirrored from home: Open/Closed/Resolved.
enum State { Open, Closed, Resolved }
State public marketState;
uint8 public winningOutcome; // set on resolution

/// @notice Total collateral escrowed for all users.
uint256 public totalEscrowed;

event IntentSubmitted(
    uint256 indexed nonce,
    address indexed user,
    uint8 outcomeIndex,
    uint256 collateralIn
);
event FillConfirmed(
    uint256 indexed nonce,
    uint256 sharesOut,
    uint256 actualPrice
);
event IntentRefunded(uint256 indexed nonce, uint256 collateralRefunded);
event WrappedPositionMinted(
    address indexed user,
    uint8 outcomeIndex,
    uint256 amount
);
event MarketResolved(uint8 winningOutcome);
event Redeemed(address indexed user, uint8 outcomeIndex, uint256 collateral);

error WrongState(State current, State required);
error InvalidOutcome();
error IntentNotPending(uint256 nonce);
error Unauthorized();
error ZeroCollateral();
error InsufficientEscrow();

constructor(
    uint256 _homeChainId,
    uint256 _homeMarketId,
    address _collateral,
    address _bridgeAdapter,
    address _homeGateway
) {
    homeChainId = _homeChainId;
    homeMarketId = _homeMarketId;
    collateral = IERC20(_collateral);
    bridgeAdapter = IBridgeAdapter(_bridgeAdapter);
    homeGateway = _homeGateway;
    marketState = State.Open;

    // Deploy wrapped outcome tokens (ERC-20 representing home-held shares)
    wrappedOutcome[0] = new WrappedOutcomeToken("Wrapped YES", "wYES");
    wrappedOutcome[1] = new WrappedOutcomeToken("Wrapped NO", "wNO");
}

/// @notice Submit a cross-chain buy intent.

```

```

/// @param outcomeIndex 0 = YES, 1 = NO.
/// @param collateralIn Amount of local collateral to spend.
/// @param minSharesOut Minimum shares to accept (slippage bound); 0 = no guard.
/// @return nonce Intent nonce for tracking confirmation.
function buy(uint8 outcomeIndex, uint256 collateralIn, uint256 minSharesOut)
    external
    nonReentrant
    returns (uint256 nonce)
{
    if (marketState != State.Open) revert WrongState(marketState, State.Open);
    if (outcomeIndex > 1) revert InvalidOutcome();
    if (collateralIn == 0) revert ZeroCollateral();

    // 1. Lock collateral in escrow
    collateral.safeTransferFrom(msg.sender, address(this), collateralIn);
    totalEscrowed += collateralIn;

    nonce = _nextIntentNonce++;

    // 2. Record pending intent
    pendingIntents[nonce] = PendingIntent({
        user: msg.sender,
        outcomeIndex: outcomeIndex,
        collateralLocked: collateralIn,
        minSharesOut: minSharesOut,
        timestamp: block.timestamp
    });

    // 3. Emit bridge message (BuyIntent)
    bytes memory message = abi.encode(
        "BuyIntent",
        nonce,
        homeMarketId,
        outcomeIndex,
        collateralIn,
        minSharesOut
    );
    bridgeAdapter.sendMessage(homeChainId, homeGateway, message);

    emit IntentSubmitted(nonce, msg.sender, outcomeIndex, collateralIn);
}

/// @notice Callback from the bridge adapter when the home gateway confirms fill.
/// Only the registered bridgeAdapter may call this.
/// @param nonce The intent nonce.
/// @param sharesOut Number of outcome shares acquired on the home chain.
/// @param actualPrice The effective price paid (for UI display).
function confirmFill(uint256 nonce, uint256 sharesOut, uint256 actualPrice)
    external
    nonReentrant
{
    if (msg.sender != address(bridgeAdapter)) revert Unauthorized();

    PendingIntent memory intent = pendingIntents[nonce];
    if (intent.user == address(0)) revert IntentNotPending(nonce);
}

```

```

// Slippage check (home-side should enforce, but double-check)
if (intent.minSharesOut > 0 && sharesOut < intent.minSharesOut) {
    // Refund path (home should have signaled refund instead)
    _refund(nonce, intent);
    return;
}

// 4. Mint wrapped position receipt to user
wrappedOutcome[intent.outcomeIndex].mint(intent.user, sharesOut);

delete pendingIntents[nonce];

emit FillConfirmed(nonce, sharesOut, actualPrice);
emit WrappedPositionMinted(intent.user, intent.outcomeIndex, sharesOut);
}

/// @notice Callback when home market cannot fill (bounds miss, closed, etc.).
function confirmRefund(uint256 nonce) external nonReentrant {
    if (msg.sender != address(bridgeAdapter)) revert Unauthorized();

    PendingIntent memory intent = pendingIntents[nonce];
    if (intent.user == address(0)) revert IntentNotPending(nonce);

    _refund(nonce, intent);
}

function _refund(uint256 nonce, PendingIntent memory intent) private {
    totalEscrowed -= intent.collateralLocked;
    collateral.safeTransfer(intent.user, intent.collateralLocked);
    delete pendingIntents[nonce];
    emit IntentRefunded(nonce, intent.collateralLocked);
}

/// @notice Mirror home market state (called via bridge after home close/resolve).
function updateMarketState(State newState, uint8 _winningOutcome)
    external
    nonReentrant
{
    if (msg.sender != address(bridgeAdapter)) revert Unauthorized();
    marketState = newState;
    if (newState == State.Resolved) {
        winningOutcome = _winningOutcome;
        emit MarketResolved(_winningOutcome);
    }
}

/// @notice Redeem wrapped position receipts after resolution.
///         Burn receipt → release local collateral at 1:1 (if winning) or 0 (if losing).
/// @param outcomeIndex 0 = YES, 1 = NO.
/// @param amount       Number of wrapped shares to redeem.
function redeem(uint8 outcomeIndex, uint256 amount) external nonReentrant {
    if (marketState != State.Resolved) revert WrongState(marketState, State.Resolved);
    if (outcomeIndex > 1) revert InvalidOutcome();

    // Burn wrapped receipt
    wrappedOutcome[outcomeIndex].burn(msg.sender, amount);
}

```

```

// Payout: 1:1 if winning, 0 if losing
uint256 payout = 0;
if (outcomeIndex == winningOutcome) {
    payout = amount; // each winning share redeems for 1 collateral unit
}

if (payout > 0) {
    if (totalEscrowed < payout) revert InsufficientEscrow();
    totalEscrowed -= payout;
    collateral.safeTransfer(msg.sender, payout);
}

emit Redeemed(msg.sender, outcomeIndex, payout);
}

/// @notice Sell flow: burn wrapped receipt, emit SellIntent, await collateral return.
/// (Implementation omitted for brevity; mirrors buy flow in reverse.)
function sell(uint8 outcomeIndex, uint256 sharesIn, uint256 minCollateralOut)
    external
    returns (uint256 nonce)
{
    // Lock: burn wrapped receipt, emit SellIntent, await home AMM sell confirmation
    revert("Sell flow deferred to post-MVP");
}

}

/// @title WrappedOutcomeToken
/// @notice ERC-20 representing outcome tokens held on the home chain.
contract WrappedOutcomeToken is ERC20 {
    address public immutable proxy;

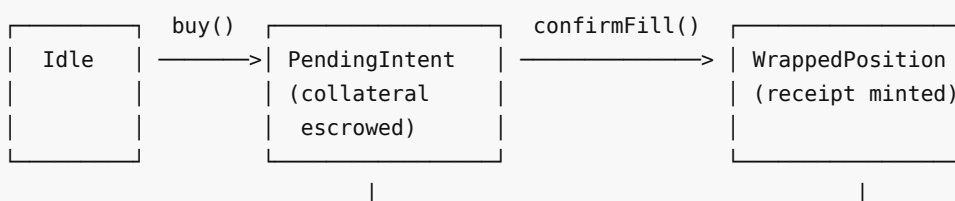
    constructor(string memory name, string memory symbol) ERC20(name, symbol) {
        proxy = msg.sender;
    }

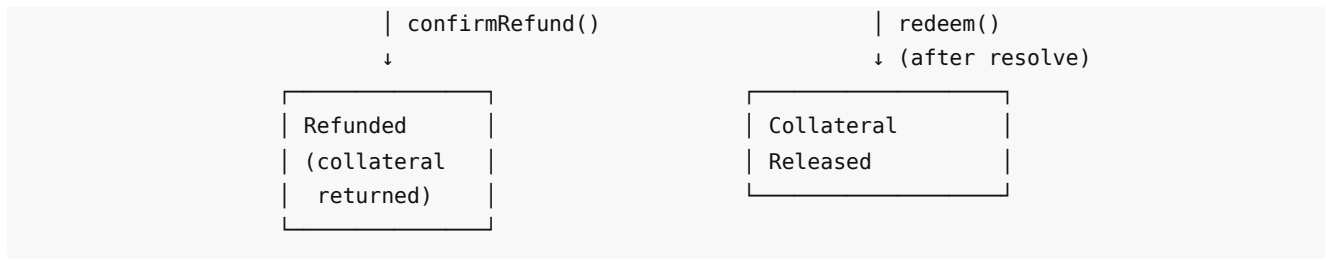
    function mint(address to, uint256 amount) external {
        require(msg.sender == proxy, "Only proxy can mint");
        _mint(to, amount);
    }

    function burn(address from, uint256 amount) external {
        require(msg.sender == proxy, "Only proxy can burn");
        _burn(from, amount);
    }
}

```

2.3 Lifecycle states





3. HomeMarketGateway — home-side receiver

3.1 Purpose

The **HomeMarketGateway** sits on the **home chain** (e.g. Arc) and:

- Receives BuyIntent / SellIntent messages from remote proxies via **IBridgeAdapter** .
- Executes the trade against the **existing** MarketAMM (the same AMM that native home-chain users trade against).
- Holds outcome tokens on behalf of the proxy (custodian role).
- Sends FillConfirmation or RefundNotification back to the originating proxy.

3.2 Illustrative Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import "./MarketAMM.sol";
import "./OutcomeToken.sol";
import "./IBridgeAdapter.sol";

/// @title HomeMarketGateway
/// @notice Receives cross-chain buy/sell intents from MarketProxy instances on
///         remote chains, executes them against the native MarketAMM, and
///         confirms fills back to the originating proxy.
///
///         This is the OffRamp/receiver analog in Chainlink CCIP terms: where
///         CCIP's OffRamp delivers messages to a target contract on the
///         destination chain, HomeMarketGateway receives intents on the home
///         chain and executes them against the unified AMM pool.
contract HomeMarketGateway {
    /// @notice Per-market AMM address (resolved from MarketFactory at deploy or runtime).
    mapping(uint256 => address) public marketAMMs;

    /// @notice OutcomeToken contract (home-chain ERC-1155).
    OutcomeToken public immutable outcomeToken;

    /// @notice Bridge adapter used to send confirmations back to proxies.
    IBridgeAdapter public immutable bridgeAdapter;

    /// @notice Custodian balances: per (remoteChainId, proxyAddress, marketId, outcomeIndex).
    ///         outcome tokens held on behalf of remote proxies.
    mapping(uint256 => mapping(address => mapping(uint256 => mapping(uint8 => uint256))))
        public custodianBalances;

    event IntentReceived(
        uint256 indexed remoteChainId,
        address indexed remoteProxy,
        uint256 indexed nonce,
```

```

        uint256 marketId,
        uint8 outcomeIndex,
        uint256 collateralIn
    );
    event IntentExecuted(
        uint256 indexed nonce,
        uint256 sharesOut,
        uint256 actualPrice
    );
    event IntentRefunded(uint256 indexed nonce, string reason);

    error MarketNotRegistered(uint256 marketId);
    error InvalidOutcome();
    error Unauthorized();

    constructor(address _outcomeToken, address _bridgeAdapter) {
        outcomeToken = OutcomeToken(_outcomeToken);
        bridgeAdapter = IBridgeAdapter(_bridgeAdapter);
    }

    /// @notice Register a market AMM (called by admin or factory).
    function registerMarket(uint256 marketId, address amm) external {
        // Access control omitted for brevity (FACTORY_ROLE in production)
        marketAMMs[marketId] = amm;
    }

    /// @notice Callback from bridge adapter when a BuyIntent arrives.
    /// @param remoteChainId The originating chain (Base, Polygon, etc.).
    /// @param remoteProxy The MarketProxy that emitted the intent.
    /// @param message ABI-encoded BuyIntent (nonce, marketId, outcome, collateral,
    minShares).
    function receiveMessage(
        uint256 remoteChainId,
        address remoteProxy,
        bytes calldata message
    ) external {
        if (msg.sender != address(bridgeAdapter)) revert Unauthorized();

        (
            string memory intentType,
            uint256 nonce,
            uint256 marketId,
            uint8 outcomeIndex,
            uint256 collateralIn,
            uint256 minSharesOut
        ) = abi.decode(message, (string, uint256, uint256, uint8, uint256, uint256));

        require(
            keccak256(bytes(intentType)) == keccak256(bytes("BuyIntent")),
            "Unknown intent type"
        );

        emit IntentReceived(remoteChainId, remoteProxy, nonce, marketId, outcomeIndex,
        collateralIn);

        address ammAddress = marketAMMs[marketId];

```

```

    if (ammAddress == address(0)) {
        _sendRefund(remoteChainId, remoteProxy, nonce, "Market not registered");
        return;
    }

    // Execute buy against the unified AMM
    MarketAMM amm = MarketAMM(ammAddress);

    // NOTE: collateralIn was locked on the remote chain; we are executing
    // a *virtual* buy here – the gateway must hold equivalent collateral
    // staked by the protocol or bridged via CCTP (see doc 04).
    // For this design, assume the gateway has approved collateral to the AMM.

    uint256 sharesOut;
    try amm.buy(outcomeIndex, collateralIn, minSharesOut) returns (uint256 shares) {
        sharesOut = shares;
    } catch {
        _sendRefund(remoteChainId, remoteProxy, nonce, "AMM buy failed");
        return;
    }

    // Hold outcome tokens in custody for the remote proxy
    uint256 tokenId = outcomeToken.encodeId(marketId, outcomeIndex);
    custodianBalances[remoteChainId][remoteProxy][marketId][outcomeIndex] += sharesOut;

    uint256 actualPrice = (collateralIn * 1e6) / sharesOut; // price in collateral per share

    emit IntentExecuted(nonce, sharesOut, actualPrice);

    // Send FillConfirmation back to proxy
    bytes memory confirmation = abi.encode(
        "FillConfirmation",
        nonce,
        sharesOut,
        actualPrice
    );
    bridgeAdapter.sendMessage(remoteChainId, remoteProxy, confirmation);
}

function _sendRefund(
    uint256 remoteChainId,
    address remoteProxy,
    uint256 nonce,
    string memory reason
) private {
    emit IntentRefunded(nonce, reason);
    bytes memory refundMsg = abi.encode("RefundNotification", nonce);
    bridgeAdapter.sendMessage(remoteChainId, remoteProxy, refundMsg);
}

/// @notice After resolution, burn custodian-held tokens and signal proxies.
/// (Full resolution propagation covered in doc 05.)
function propagateResolution(uint256 marketId, uint8 winningOutcome) external {
    // Emit resolution message to all known proxies for this market
    // (proxy registry omitted for brevity)

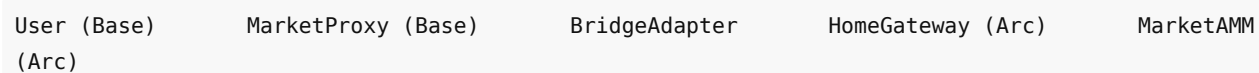
```

4. End-to-end sequence diagrams

4.1 Cross-chain BUY happy path



4.2 Bounds-miss refund path



--buy(YES, 100) --->			
minShares=98 --lock 100 USDC			
--sendMessage(BuyIntent) -->			
	--transport----->		
		--buy(YES,100,98) ----->	
			--
revert SlippageExceeded			
		<--revert-----	
		--_sendRefund----->	
		<--RefundNotif-----	
<---confirmRefund---	<---transport-----		
	--unlock 100 USDC		
	--transfer back to user		

4.3 Cross-chain SELL and redemption

User (Base)	MarketProxy (Base)	BridgeAdapter	HomeGateway (Arc)	MarketAMM
After resolution:				
	<--updateMarketState----- <--ResolutionMsg---			
	(Resolved, YES wins)			
--redeem(YES,95)-->				
	--burn wYES=95			
	--payout = 95 * 1 = 95			
<--95 USDC-----				

5. The wrapped position receipt

5.1 What it represents

A **wrapped position receipt** (e.g. `wYES` , `wNO`) is an **ERC-20 token** issued by the `MarketProxy` on the remote chain that represents **outcome tokens held in custody on the home chain**. Each wrapped token is backed 1:1 by outcome shares held in `HomeMarketGateway.custodianBalances` .

Property	Value
Standard	ERC-20 (simplicity; fungible within outcome)
Mint	When <code>confirmFill()</code> is called after home AMM executes the buy
Burn	On <code>redeem()</code> (after resolution) or <code>sell()</code> (initiate sell-back to home AMM)
Transferable	Yes — users can trade wrapped receipts peer-to-peer on the remote chain
Redeemable	After market resolution, 1 wYES (if YES wins) → 1 collateral unit; losing outcome → 0

5.2 Why ERC-20 instead of mirroring ERC-1155

- **Simpler integration** with remote-chain DeFi (AMMs, lending, wallets all handle ERC-20).
- **Clear semantic:** one token contract per (proxy, outcome) — no token-ID encoding needed on the remote chain.

- **Gas efficiency:** ERC-20 transfers are cheaper than ERC-1155 on many chains.

5.3 Safety: "as safe as the weakest link"

A wrapped position is **as safe as min(home chain security, bridge adapter security)**:

- If the **home chain** is compromised (51% attack, contract exploit), the custodian balances on `HomeMarketGateway` can be drained → wrapped receipts lose backing.
- If the **bridge adapter** is compromised (forged messages), an attacker can mint unbacked wrapped receipts or steal escrowed collateral.
- If **both** are secure, the wrapped position is fully backed and redeemable at resolution.

Mitigation (per whitepaper §5.2 and doc 06):

- Allowlist only audited, high-reputation bridge adapters (Chainlink CCIP as the reference).
- Per-route exposure caps (e.g. max 1M USDC escrowed per proxy).
- Circuit breaker pausing new intents (never blocking redemptions) on anomaly detection.

6. Accounting invariants

6.1 Collateral conservation

Home-side invariant:

```
Σ(custodianBalances across all proxies for market M, outcome 0)
  * 1 collateral per share
== collateral staked in HomeGateway for market M, outcome 0
```

The home gateway must hold (or have bridged via CCTP — see [doc 04](#)) sufficient collateral to back every custodian-held outcome token.

Proxy-side invariant:

```
totalEscrowed == Σ(pendingIntents[i].collateralLocked)
                + Σ(wrapped receipts outstanding * redemption value after resolution)
```

Every unit of local collateral locked in the proxy is either:

- Pending fill (escrowed in `PendingIntent`), or
- Backing a wrapped receipt (redeemable 1:1 if winning, 0 if losing).

6.2 No double-spend across chains

Each intent is assigned a **unique nonce** per proxy. The home gateway processes each nonce exactly once:

- On fill: custodian balance incremented, `FillConfirmation` sent.
- On refund: `RefundNotification` sent, proxy releases escrow.

Replay protection:

- Bridge adapters must enforce message uniqueness (CCIP does this via sequence numbers).
- Home gateway rejects duplicate nonces (not shown in illustrative code; production must check).

6.3 Idempotency and reorg safety

- **Idempotency:** if a confirmation message is delivered twice (rare but possible with some bridges), the proxy's `confirmFill()` must check `pendingIntents[nonce]` exists and delete it after processing → second call reverts with `IntentNotPending`.
- **Reorg on home chain:** if the home AMM buy is reorged out, the gateway must re-execute or refund. This is a bridge-adapter concern — CCIP's finality guarantees minimize this risk.

7. Local-first routing (bypass for native users)

From [whitepaper §5.2](#):

If a market's home chain is the user's chain, the proxy path is bypassed entirely — native speed, no bridge risk.

7.1 Implementation strategy

The **frontend** (Next.js + wagmi) checks:

```
const userChainId = useChainId();
const marketHomeChainId = market.homeChainId;

if (userChainId === marketHomeChainId) {
  // Direct route: call MarketAMM.buy() on the PredictionMarket
  const tx = await marketAMM.buy(outcomeIndex, collateralIn, minSharesOut);
} else {
  // Proxy route: call MarketProxy.buy() on the user's current chain
  const proxyAddress = getMarketProxy(marketHomeChainId, market.id, userChainId);
  const tx = await marketProxy.buy(outcomeIndex, collateralIn, minSharesOut);
}
```

7.2 UX benefit

- **Instant finality:** no bridge latency (seconds to minutes) — trade confirms in one block.
- **No bridge trust:** position is a native ERC-1155 OutcomeToken, not a wrapped receipt.
- **Lower cost:** no bridge message fee, no CCTP gas.

This is a **first-class optimization** — not an edge case. On Arc (the strategic home for high-value markets per [whitepaper §4.1](#)), all Arc-native users trade at native speed.

8. Chainlink CCIP architectural inspiration

8.1 The explicit parallel

Chainlink CCIP separates:

- **Router (source chain):** user-facing entry point; emits cross-chain messages.
- **OnRamp (source chain):** lane-specific sender; queues messages, charges fees, forwards to DON.
- **DON (Decentralized Oracle Network):** transports messages cross-chain with cryptographic attestation.
- **OffRamp (destination chain):** lane-specific receiver; verifies DON signatures, delivers messages to target contract.
- **Risk Management Network (RMN):** independent monitors that can pause lanes on anomaly detection.

Justify's bridge layer mirrors this:

CCIP component	Justify analog
Router	MarketProxy (user-facing, emits intents)
OnRamp	IBridgeAdapter.sendMessage() on source chain (lane abstraction)
DON + OffRamp	IBridgeAdapter on destination chain + delivery to HomeMarketGateway
Target contract	HomeMarketGateway (receives, executes, confirms)
RMN	Circuit breaker in MarketProxy / allowlist governance (doc 06)

8.2 Why this matters

1. **Proven architecture:** CCIP secures billions in cross-chain value; adopting its separation of concerns reduces novel surface area.
2. **Adapter seam:** by modeling `IBridgeAdapter` after CCIP's interface, we can plug in CCIP itself as the reference transport (see [doc 02](#)) or alternative bridges (LayerZero, Wormhole) as adapters.
3. **Forward compatibility:** when Chainlink launches new features (e.g. programmable token transfers, verifiable compute triggers), Justify can adopt them via adapter upgrades without redesigning the proxy/home split.

Acknowledgment in code: every bridge-related contract includes a header comment noting the Chainlink CCIP inspiration.

9. Integration checklist (per remote chain)

To deploy a `MarketProxy` for market M on remote chain R:

1. **Deploy `MarketProxy`** with:
 - `homeChainId` = M's home chain (e.g. Arc = 5042002).
 - `homeMarketId` = M's market ID on the home chain.
 - `collateral` = local USDC address on chain R.
 - `bridgeAdapter` = the deployed `CCIPBridgeAdapter` (or other) on chain R (see [doc 02](#)).
 - `homeGateway` = the `HomeMarketGateway` address on the home chain.
2. **Register the proxy** in `HomeMarketGateway.proxyRegistry` (not shown in illustrative code; production must track which proxies exist for each market).
3. **Fund the bridge adapter** with native gas tokens (e.g. ETH on Base) for CCIP message fees.
4. **Seed collateral** in `HomeMarketGateway` (or rely on CCTP to transfer USDC per intent — see [doc 04](#)).
5. **Allowlist the route** in protocol governance (chain R ↔ home chain for market M).

10. Open design questions and future work

Question	Status	Notes
Sell flow	Deferred	Sell = burn wrapped receipt, emit <code>SellIntent</code> , home AMM sells, gateway returns collateral. Mirrors buy; omitted for MVP (whitepaper §15 item 4).
CCTP integration	Planned (doc 04)	Use Circle CCTP to bridge USDC legs instead of staking collateral in <code>HomeGateway</code> . Reduces capital lock.
Proxy upgrade	Designed	Proxies are non-upgradeable (no storage corruption risk); new market versions deploy fresh proxies. Old proxies remain live until their markets resolve.
Gas strategy	Research	Who pays for the home-side <code>amm.buy()</code> gas? Options: (1) pre-paid by protocol, (2) embedded in bridge fee, (3) user-pays via CCIP programmable token transfers.
Partial fills	Future	Current design: all-or-nothing fill within slippage bounds. Future: accept partial fill, refund remainder (requires two-phase commit on home gateway).

11. Cross-references

- [doc 01: IBridgeAdapter interface](#) — the transport seam this design consumes.
- [doc 02: Chainlink CCIP adapter](#) — the reference implementation of `IBridgeAdapter`.
- [doc 04: Per-chain bridge playbook](#) — how to configure lanes, CCTP for USDC, gas funding.
- [doc 05: Cross-chain resolution](#) — settlement propagation from home to all proxies.
- [Whitepaper §5](#) — the product-level view this design implements.

- [contracts/src/MarketAMM.sol](#) — the unified AMM pool the home gateway executes against.
-

END OF DOCUMENT

This design ensures that Justify markets achieve **unified liquidity** (one pool, one price) and **single source of truth** (one home-chain state) across all supported blockchains, while preserving **local-first speed** for native users and **explicit acknowledgment** of the Chainlink CCIP architectural inspiration that makes the bridge layer robust and forward-compatible.

Per-Chain Bridge Playbook — operational guide to lanes, selectors, collateral, and USDC legs

Bridge layer operational how-to for Justify. This document describes:

1. How to stand up a bridge on each target chain
2. Per-chain infrastructure facts (CCIP selectors, USDC addresses, lane availability)
3. Route/lane configuration model
4. USDC value transfer via Circle CCTP
5. Environment variable catalog (additive, mirrors arc-layer-plan.md)

Chainlink CCIP foundation. This playbook is built around **Chainlink CCIP** as the reference cross-chain transport, and the route/lane model is **inspired by Chainlink CCIP lanes**.

1. Add a Chain — operational checklist

Standing up bridge support on a new chain extends whitepaper §4.2 ("deploy + register artifact + fund treasury") with bridge-specific steps. The choice is binary: deploy the full home stack OR deploy a MarketProxy.

1.1 Home chain deployment

If the chain will **host home markets** (hold collateral pools, run AMM pricing, and settle):

- ```
[] 1. Deploy the full Justify contract stack
 - MarketFactory, PredictionMarket, MarketAMM, OutcomeToken,
 OracleResolver, FeeTreasury, AccessControl
 - See arc-layer-plan.md for the config-and-redeploy model

[] 2. Inject collateral token address
 - MarketFactory constructor takes _collateral as a parameter
 - Use canonical USDC where available (see §2 matrix)
 - Fallback: deploy MockUSDC for testing

[] 3. Deploy IBridgeAdapter (from 02-chainlink-ccip-adapter.md)
 - ChainlinkCCIPAdapter(ccipRouterAddress, linkTokenAddress)
 - Verify CCIP router address from Chainlink docs per chain

[] 4. Register deployment artifact
 - Generate contracts/deployments/{chain}.json with chainId,
 contract addresses, ABIs
 - Backend serves this at runtime (no rebuild per deploy)

[] 5. Fund fee accounts
 - FeeTreasury: seed with collateral for operational reserve
 - CCIP adapter: fund with LINK for cross-chain message fees
 - Deployer account: acquire gas token + collateral from faucet/bridge

[] 6. Configure access control
 - Grant FACTORY_ROLE to MarketFactory
 - Grant RESOLVER_ROLE to OracleResolver (or CRE service account)
 - Grant TREASURY_ROLE to fee-withdrawal operator
```

### 1.2 Remote/proxy chain deployment

If the chain will **proxy to a home chain elsewhere** (forward orders via bridge):

- [ ] 1. Deploy MarketProxy contract (from 03-marketproxy-and-home-market.md)
  - MarketProxy(homeChainId, homeMarketAddress, localCollateralToken, bridgeAdapter)
  - homeChainId and homeMarketAddress point to the actual AMM
- [ ] 2. Deploy IBridgeAdapter for this route
  - Same ChainlinkCCIPAdapter as home side
  - CCIP router address for *\*this\** chain (source side)
- [ ] 3. Configure the lane route (source → home)
  - Set CCIP chain selector for destination (home chain)
  - Allowlist the route in MarketProxy (owner-only call)
  - Per-route exposure cap (max collateral locked awaiting confirmation)
- [ ] 4. Fund LINK for outbound CCIP messages
  - MarketProxy pays CCIP fees on every Buy/Sell bridged intent
  - Monitor LINK balance; replenish via keeper or manual top-up
- [ ] 5. Configure collateral token
  - If USDC and both chains support CCTP: flag for burn-and-mint (see §4)
  - Otherwise: CCIP token pool or lock-and-mint
- [ ] 6. Register the proxy in the artifact registry
  - Backend indexes MarketProxy events (IntentSent, FillConfirmed)
  - UI shows "bridged" badge and estimated fill time
- [ ] 7. Test the round trip
  - Place a small Buy on the proxy, confirm fill from home AMM
  - Verify wrapped receipt minted on proxy chain
  - Redeem after home market resolves

### 1.3 Allowlist and exposure limits

Every (sourceChain → homeChain) lane must be **explicitly allowlisted** by the MarketProxy owner. This prevents griefing via unsupported routes.

```
// MarketProxy owner call
function allowlistRoute(
 uint256 homeChainId,
 uint64 ccipChainSelector,
 uint256 maxExposure
) external onlyOwner;
```

- homeChainId : EVM chain ID of the home market's chain
- ccipChainSelector : Chainlink CCIP selector for the home chain (distinct from chain ID; see §2)
- maxExposure : max collateral locked in pending fills; excess orders revert

## 2. Per-Chain Infrastructure Facts

One row per target network. **Note on CCIP selectors:** these are **distinct from EVM chain IDs** and are assigned by Chainlink. Where values are uncertain, we mark "confirm on Chainlink registry" rather than inventing data.

| Network | Chain ID | CCIP Chain | Canonical USDC | CCTP Support |
|---------|----------|------------|----------------|--------------|
|         |          |            |                |              |

|                                |         | Selector                                                           |                                                                                                     |                                                             |
|--------------------------------|---------|--------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| <b>Arc Testnet</b><br>(Circle) | 5042002 | confirm on Chainlink docs                                          | Circle USDC (native 18-dec gas token; confirm ERC-20 address)                                       | Unknown (Arc is new; confirm with Circle)                   |
| <b>Base</b>                    | 8453    | confirm on Chainlink docs (Base is a known CCIP chain)             | 0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913 (6 decimals)                                             | <b>Yes</b> — Base is a native USD chain                     |
| <b>Ethereum</b>                | 1       | confirm on Chainlink docs (Ethereum mainnet is a known CCIP chain) | 0xA0b86991c6218b36c1d19D4a2e9Eb0cE3606eB48 (6 decimals)                                             | <b>Yes</b> — Ethereum is the CCTP                           |
| <b>Polygon PoS</b>             | 137     | confirm on Chainlink docs (Polygon is a known CCIP chain)          | 0x3c499c542cEF5E3811e1192ce70d8cC03d5c3359 (6 decimals, native USDC; old bridged USDC.e deprecated) | <b>Yes</b> — Polygon native USDC 2023                       |
| <b>BNB Smart Chain</b>         | 56      | confirm on Chainlink docs (BSC is a known CCIP chain)              | confirm canonical USDC or USDT address (BSC ecosystem uses both)                                    | Confirm (CCTP primarily Ethereum/Base/Polygon/A as of 2025) |

**Action for implementer:** Before deploying to a chain, confirm these values from:

- [Chainlink CCIP supported networks](#) — chain selectors, router addresses
- [Circle CCTP supported domains](#) — USDC addresses, CCTP availability
- Chain explorer (Etherscan, Basescan, Polygonscan, etc.) — verify token contract addresses

## 2.1 Why CCIP chain selectors are distinct from chain IDs

Chainlink CCIP assigns a unique `uint64` selector to each supported network. This selector is used in CCIP router calls and lane configuration, and is **not the same as the EVM chain ID**. Example (hypothetical, confirm from Chainlink):

- Ethereum mainnet: chain ID `1`, CCIP selector might be `1` or another assigned value

- Base: chain ID 8453 , CCIP selector assigned by Chainlink
- Polygon: chain ID 137 , CCIP selector assigned by Chainlink

The selector is the routing key within CCIP's off-chain network; the chain ID is the on-chain identifier. Both must be configured correctly.

### 3. Route and Lane Configuration

A **route** is a (sourceChain → homeChain) pair. A **lane** is Chainlink CCIP's term for a supported source→destination path. Justify routes map onto CCIP lanes.

#### 3.1 Routing table model

The backend maintains a routing table (Postgres or served artifact):

```
{
 "routes": [
 {
 "sourceChain": { "id": 8453, "name": "Base" },
 "homeChain": { "id": 5042002, "name": "Arc" },
 "ccipSelector": "confirm_arc_selector",
 "adapterAddress": "0x...",
 "maxExposure": "10000000000",
 "status": "active"
 },
 {
 "sourceChain": { "id": 137, "name": "Polygon" },
 "homeChain": { "id": 1, "name": "Ethereum" },
 "ccipSelector": "confirm_eth_selector",
 "adapterAddress": "0x...",
 "maxExposure": "50000000000",
 "status": "active"
 }
]
}
```

- Each route has its own CCIP adapter instance (or shares one if the adapter is stateless per lane).
- The frontend reads this table to show users "This market is on Arc; you can trade from Base via bridge (estimated 2-5 min fill)".

#### 3.2 Market home assignment

Every PredictionMarket is created on exactly one home chain. The MarketFactory emits:

```
event MarketCreated(
 uint256 indexed marketId,
 address marketAddress,
 uint256 homeChainId,
 ...
);
```

The backend indexes homeChainId so it can route proxy orders to the correct home AMM.

#### 3.3 Per-route adapter selection

Different routes may use different bridge transports:

- **Arc ↔ Base, Base ↔ Ethereum, Polygon ↔ Ethereum:** Chainlink CCIP adapter (fast, decentralized, LINK-fee)
- **Testnet routes:** could use a mock adapter for integration tests
- **Future:** LayerZero adapter, native bridge adapter, or custom relay

The `IBridgeAdapter` interface (from 02-chainlink-ccip-adapter.md) abstracts this. `MarketProxy` calls `adapter.sendIntent()` and doesn't care about the underlying transport.

## 4. USDC Value Legs via Circle CCTP

When the bridged collateral is **USDC** and both source and home chains support **Circle CCTP** (Cross-Chain Transfer Protocol), use CCTP for the **value leg** (the actual USDC transfer) while CCIP carries the **intent message** (Buy/Sell order metadata).

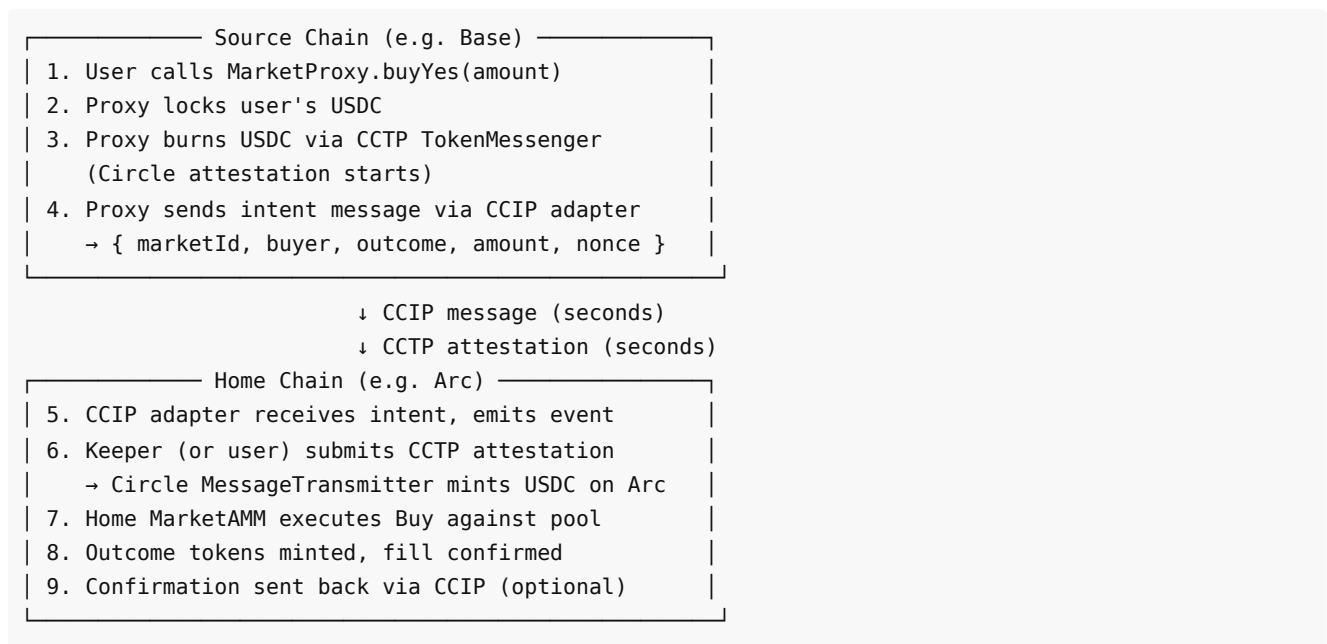
### 4.1 CCTP burn-and-mint model

Circle CCTP is a native USDC bridge:

1. **On source chain:** user's USDC is **burned** (not locked) via Circle's `TokenMessenger` contract.
2. **Off-chain:** Circle's attestation service observes the burn and signs an attestation.
3. **On home chain:** the attestation is submitted to Circle's `MessageTransmitter`, which **mints** native USDC on the destination.

Result: the USDC that arrives on the home chain is **native USDC**, not a wrapped/bridged token. No liquidity pool needed, no bridge TVL risk (beyond Circle custody), and no divergence from \$1 peg.

### 4.2 CCIP + CCTP dual-leg flow



**Key coordination:** the home-side keeper must wait for **both** the CCIP intent message AND the CCTP attestation before calling `MarketAMM.buy()`. If CCIP arrives first (likely), the intent is queued; when CCTP USDC mints, the fill executes.

### 4.3 CCTP vs CCIP token pools

Chainlink CCIP also supports token transfers via **CCIP token pools** (lock-and-mint or burn-and-mint administered by Chainlink). For USDC:

- **Use CCTP when available** — it's Circle's native bridge, cheaper fee (no LINK fee for USDC leg), and settles in native USDC.
- **Use CCIP token pool as fallback** — if one chain doesn't support CCTP, or for non-USDC collateral (e.g. DAI, USDT).

The `IBridgeAdapter` implementation chooses which path:

```
function sendIntent(
 IntentMessage memory intent,
 address collateralToken,
 uint256 amount
) external payable returns (bytes32 messageId) {
 if (collateralToken == USDC && cctpSupported(destChain)) {
 _burnViaUSDC(amount); // Circle CCTP burn
 _sendIntentMessage(intent); // CCIP message-only
 } else {
 _sendIntentWithTokens(intent, collateralToken, amount); // CCIP token pool
 }
}
```

#### 4.4 Native-USDC chains

Per the whitepaper §4.1 and the matrix above:

- **Arc, Base, Ethereum, Polygon:** native Circle USDC, CCTP-capable (confirm Arc with Circle).
- **BSC:** USDC exists but CCTP support is TBD; may need CCIP token pool or BSC-specific bridge.

### 5. Bridge Layer Environment Variables

Additive to the base stack (from arc-layer-plan.md). Per chain, per route.

#### 5.1 Home chain env (where markets live)

```
Existing (from arc-layer-plan.md)
NEXT_PUBLIC_CHAIN_ID=5042002
NEXT_PUBLIC_RPC_URL=https://rpc.testnet.arc.network
RPC_URL=https://rpc.testnet.arc.network
COLLATERAL_USDC_ADDRESS=0x... # Circle USDC on Arc

Bridge additions for home chain
CCIP_ROUTER_ADDRESS=0x... # Chainlink CCIP Router on Arc (from Chainlink docs)
CCIP_CHAIN_SELECTOR=<arc_selector> # Arc's CCIP selector (from Chainlink)
LINK_TOKEN_ADDRESS=0x... # LINK token on Arc (for CCIP fees)
CCTP_TOKEN_MESSENGER=0x... # Circle TokenMessenger on Arc (if CCTP supported)
CCTP_MESSAGE_TRANSMITTER=0x... # Circle MessageTransmitter on Arc
HOME_CHAIN_ID=5042002 # Redundant with CHAIN_ID but explicit for clarity
```

#### 5.2 Remote/proxy chain env (e.g. Base → Arc route)

```
Base chain identity
PROXY_CHAIN_ID=8453
PROXY_RPC_URL=https://mainnet.base.org
PROXY_COLLATERAL_USDC_ADDRESS=0x833589fCD6eDb6E08f4c7C32D4f71b54bdA02913

Bridge adapter on Base
PROXY_CCIP_ROUTER_ADDRESS=0x... # CCIP Router on Base
PROXY_CCIP_CHAIN_SELECTOR=<base_selector>
PROXY_LINK_TOKEN_ADDRESS=0x...
PROXY_CCTP_TOKEN_MESSENGER=0x... # Base CCTP messenger

Target home chain
```

```

HOME_CHAIN_ID=5042002 # Arc
HOME_CCIP_CHAIN_SELECTOR=<arc_selector> # Arc's CCIP selector
HOME_MARKET_FACTORY_ADDRESS=0x... # MarketFactory on Arc (from deployments/arc.json)

Bridge adapter deployment on Base
BRIDGE_ADAPTER_ADDRESS=0x... # ChainlinkCCIPAdapter on Base
MARKET_PROXY_ADDRESS=0x... # MarketProxy on Base

Route config
MAX_ROUTE_EXPOSURE=10000000000 # 10,000 USDC (6 decimals) max pending per route
CCIP_ESTIMATED_FILL_SECONDS=120 # 2 min typical; surface in UI

```

### 5.3 Multi-route env pattern

For N remote chains, the backend can namespace env vars:

```

BRIDGE_ROUTE_BASE_TO_ARC_ADAPTER=0x...
BRIDGE_ROUTE_BASE_TO_ARC_MAX_EXPOSURE=10000000000
BRIDGE_ROUTE_POLYGON_TO_ETH_ADAPTER=0x...
BRIDGE_ROUTE_POLYGON_TO_ETH_MAX_EXPOSURE=50000000000
...

```

Or load from a JSON config served at runtime (preferred for >3 routes).

## 6. Chainlink CCIP as Reference Transport

This playbook assumes **Chainlink CCIP** because:

1. **Mature, production-grade cross-chain messaging** (live on Ethereum, Base, Polygon, Arbitrum, Optimism, Avalanche, BNB, and more).
2. **Decentralized oracle network (DON)** for message relay + independent **Risk Management Network** for anomaly detection — mirrors Justify's security goals.
3. **Lane model** — Chainlink explicitly supports source→destination lanes with known latencies and fee structures; Justify routes map cleanly onto lanes.
4. **Token transfer support** — both arbitrary messages (for our intent) and token pools (fallback if CCTP unavailable).
5. **Audited and battle-tested** — Chainlink's security posture reduces bridge risk vs custom relays.

### 6.1 Chainlink CCIP lane terminology

- **OnRamp (source chain):** contract that accepts outgoing CCIP messages and tokens; emits CCIPSendRequested.
- **OffRamp (destination chain):** contract that delivers incoming messages; emits ExecutionStateChanged.
- **Router:** entry point on each chain; routes to the correct OnRamp/OffRamp per lane.
- **DON (Decentralized Oracle Network):** off-chain nodes that observe OnRamp, reach consensus, and call OffRamp on destination.
- **RMN (Risk Management Network):** independent blessing/cursing network that can halt a lane if anomaly detected.

Justify's `IBridgeAdapter` is a thin wrapper around the Router; `MarketProxy.sendIntent()` calls `adapter.sendMessage()` which calls `CCIPRouter.ccipSend()`.

### 6.2 Mapping Justify concepts to CCIP

| Justify term                             | CCIP equivalent                              |
|------------------------------------------|----------------------------------------------|
| Route (sourceChain → homeChain)          | Lane (source → destination chain)            |
| <code>IBridgeAdapter.sendIntent()</code> | <code>CCIPRouter.ccipSend()</code> on source |

|                                                         |                                                         |
|---------------------------------------------------------|---------------------------------------------------------|
| <code>IBridgeAdapter.receiveIntent()</code>             | <code>CCIPReceiver._ccipReceive()</code> on destination |
| IntentMessage (marketId, buyer, outcome, amount, nonce) | CCIP Client.EVM2AnyMessage with data payload            |
| LINK fee                                                | CCIP message fee (paid in LINK or native token)         |
| Wrapped receipt (ERC-20 on proxy chain)                 | Not a CCIP primitive; Justify-specific                  |
| Home-market fill confirmation                           | CCIP return message (optional; or event indexing)       |

## 7. Operational Runbook Summary

### 7.1 Deploying a new home chain (e.g. Arc)

1. Confirm Arc's CCIP support (Chainlink docs) → get chain selector
2. Confirm Circle USDC address on Arc (or deploy MockUSDC fallback)
3. Deploy full Justify stack with `COLLATERAL_USDC_ADDRESS` set
4. Deploy `ChainlinkCCIPAdapter(arcCCIPRouter, arcLINK)`
5. Fund deployer with Arc gas USDC from `faucet.circle.com`
6. Fund `FeeTreasury` with seed USDC
7. Fund CCIP adapter with LINK for inbound message execution (if needed)
8. Register contracts/deployments/`arc.json`
9. Seed one market, test local Buy → confirm AMM pricing works

### 7.2 Adding a proxy chain route (e.g. Base → Arc)

1. Confirm Base CCIP support → get Base's CCIP selector
2. Confirm Base→Arc lane exists in Chainlink lane explorer
3. Deploy `ChainlinkCCIPAdapter` on Base (points to Base CCIP router)
4. Deploy `MarketProxy` on Base:
  - `homeChainId` = 5042002 (Arc)
  - `homeMarketAddress` = Arc's `MarketFactory` or specific market
  - `collateralToken` = Base USDC (0x833...)
  - `bridgeAdapter` = Base CCIP adapter address
5. Call `MarketProxy.allowlistRoute(5042002, arcCCIPSelector, maxExposure)`
6. Fund Base CCIP adapter with LINK (for outbound CCIP fees)
7. If CCTP: confirm Base & Arc CCTP support, set CCTP addresses in adapter
8. Register Base deployment artifact
9. Test: place Buy on Base proxy → observe CCIP message → confirm fill on Arc → verify wrapped receipt minted on Base
10. Monitor LINK balance on Base adapter; set up auto-refill keeper

### 7.3 Circuit breaker / pausing a route

If anomaly detected (e.g. CCIP lane halted, CCTP attestation delay, home AMM drained):

```
// MarketProxy owner call
function pauseRoute(uint256 homeChainId) external onlyOwner;
```

Paused route:

- **Blocks new intents** (Buy/Sell calls revert)
- **Never blocks redemptions** (users can always redeem wrapped receipts after home market resolves)
- Unpauses after incident resolved and exposure cap restored

Mirrors Chainlink RMN's "curse" mechanism (see 06-security-and-risk-management.md).

## 8. Forward Links

- **02-chainlink-ccip-adapter.md** — the adapter implementation this playbook assumes
- **05-cross-chain-resolution.md** — how settlement propagates from home to all proxies
- **arc-layer-plan.md** — the "chain is config" foundation for per-chain deployment
- **Chainlink CCIP docs** — [docs.chain.link/ccip](https://docs.chain.link/ccip) — chain selectors, router addresses, fee calculator, lane explorer

---

**Operational honesty:** Where CCIP selectors, USDC addresses, or CCTP support are marked "confirm", the implementer **MUST** verify from authoritative sources (Chainlink docs, Circle docs, chain explorers) before deploying. This playbook provides the structure, not the constantly-changing on-chain addresses.

# Cross-chain resolution — one resolution settles all chains

How Justify markets resolve once on the home chain and propagate to every proxy chain, unifying settlement across **Arc, Base, Ethereum, Polygon, and BSC**. This design is **built on two Chainlink products working together**: Chainlink CRE (automation + off-chain compute resolving the home market) and Chainlink CCIP (carrying the resolution broadcast across chains).

---

## Table of contents

- [1. Principle](#)
  - [2. Architecture](#)
  - [3. The Chainlink foundation](#)
  - [4. Resolution flow](#)
  - [5. Idempotency and safety](#)
  - [6. Current state and future work](#)
  - [References](#)
- 

## 1. Principle

From [whitepaper §5.3](#) (Phase 3):

*Resolution executes **once, on the home chain**, and is propagated to all proxies via the same bridge adapters used for order flow — so decentralizing resolution on one chain decentralizes it everywhere at once.*

The core property: **one resolution event settles all chains**. No proxy chain can disagree with home-market state; every proxy unlocks redemption only after receiving the home resolution broadcast.

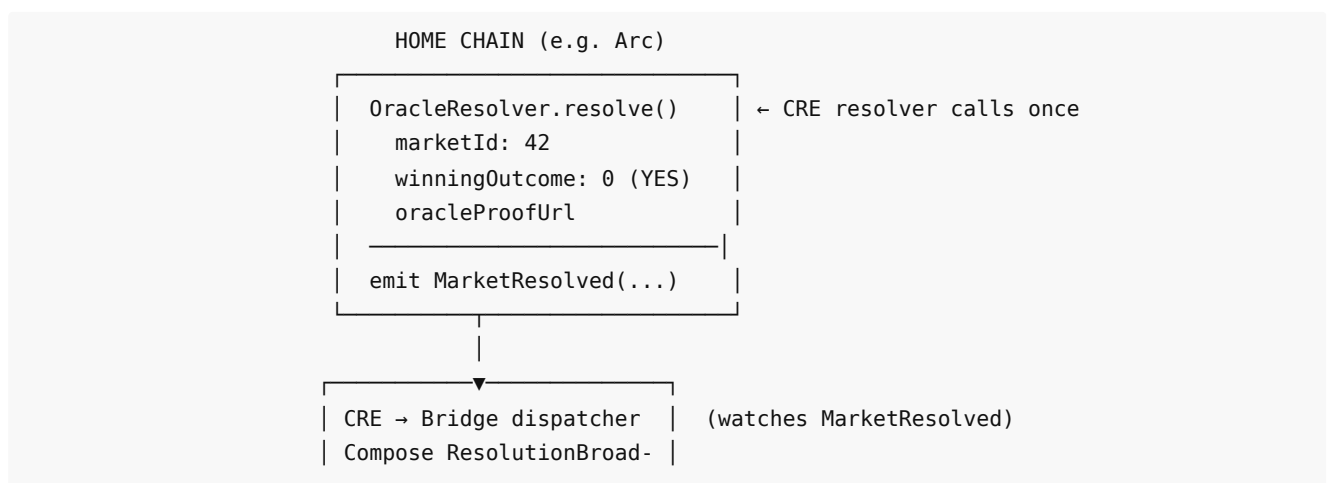
In the bridge phase:

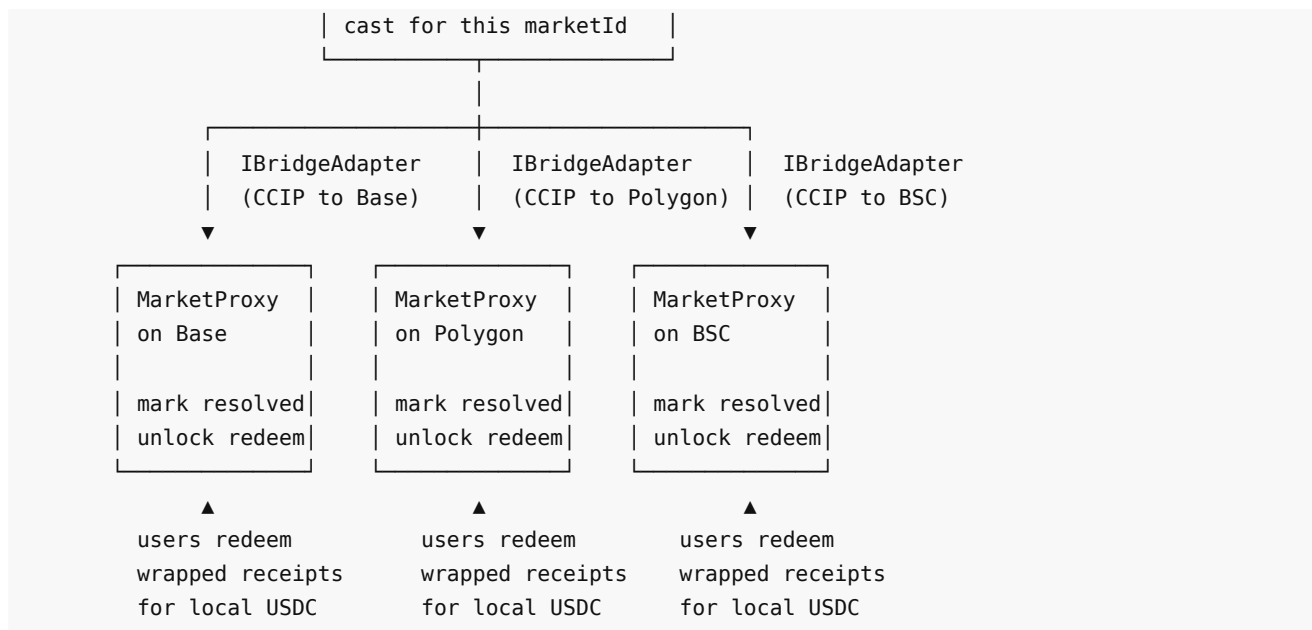
- A market has **one home chain** (e.g. Arc) where its `PredictionMarket`, AMM, and collateral pool live.
- That market has **proxy contracts** on every other supported chain (Base, Ethereum, Polygon, BSC) that forward orders to the home AMM and hold wrapped position receipts for remote traders.
- When the home market resolves, a `ResolutionBroadcast` message is sent (exactly once) via the `IBridgeAdapter` to every proxy chain.
- Each `MarketProxy` marks itself resolved and enables users to redeem their wrapped receipts for local collateral according to the home resolution outcome.

**No multi-chain coordination is needed**; proxies are stateless receivers of the home truth. The home chain is the single source of resolution authority.

---

## 2. Architecture





## Key components

| Component             | Role                                                                                                                                                                                                                                     | Lives on            |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|
| OracleResolver        | Authoritative resolver; <code>resolve(marketId, outcome, proofUrl)</code> records the winning outcome and emits <code>MarketResolved</code>                                                                                              | Home chain          |
| CRE resolver service  | Automated off-chain service (see <a href="#">cre-resolver-plan.md</a> ) that fetches price data, decides outcome, and calls <code>OracleResolver.resolve()</code> signed by the oracle key (mnemonic idx 1, <code>RESOLVER_ROLE</code> ) | Off-chain (Go)      |
| Bridge dispatcher     | Watches <code>MarketResolved</code> on home chain, composes a <code>ResolutionBroadcast</code> message, and sends it via <code>IBridgeAdapter.sendResolution(...)</code> to every known proxy chain                                      | Off-chain or keeper |
| IBridgeAdapter (CCIP) | Sends the resolution message cross-chain; first production adapter is Chainlink CCIP (see <a href="#">02-chainlink-ccip-adapter.md</a> )                                                                                                 | Per-lane contracts  |
| MarketProxy           | On each proxy chain, receives <code>ResolutionBroadcast</code> , verifies the home origin, marks itself resolved, and unlocks redemption of wrapped position receipts                                                                    | Proxy chains        |

## 3. The Chainlink foundation

This resolution architecture is **inspired by and built on Chainlink** products:

### 3.1 Chainlink CRE — resolving the home market

**Chainlink Compute / CRE** (off-chain compute, on-chain write) is the automation layer that decides market outcomes without human intervention.

From [cre-resolver-plan.md](#) :

```

price API (e.g. Coingecko, or Chainlink Data Feeds / Functions)
 | fetch
 ▼
[decision] compare price vs market.target (gt/lt) at/after resolutionAt
 | outcome = YES(0) | NO(1)
 ▼
[write] OracleResolver.resolve(marketId, outcome, proofUrl)

```

```
| signed by oracle key (RESOLVER_ROLE, mnemonic idx 1)
| emits MarketResolved(marketId, outcome, proofUrl)
```

The CRE resolver is a Go service that:

- Fetches external data (price APIs, or alternatively **Chainlink Data Feeds** for on-chain price references; Chainlink Functions for more complex compute).
- Compares the data against the market's resolution rule (e.g. "ETH > \$3000 by date Y").
- Calls `OracleResolver.resolve()` on the **HOME chain only** when the condition is met and the resolution time has passed.
- Records the `oracleProofUrl` as an audit trail (the price-API URL or Chainlink Functions request ID).

This is **automated, decentralized resolution** per whitepaper §5.3 Phase 2. The human operator is removed from the loop for objectively verifiable events.

**Key property:** CRE resolves the home market **once**. The on-chain `isResolved()` guard prevents double-resolution. The `MarketResolved` event is the trigger for the next step: cross-chain propagation.

### 3.2 Chainlink CCIP — propagating resolution cross-chain

**Chainlink CCIP** (Cross-Chain Interoperability Protocol) is the transport layer that carries the `ResolutionBroadcast` from the home chain to every proxy chain.

From [02-chainlink-ccip-adapter.md](#):

*Justify's bridge layer is explicitly and directly inspired by the architecture of Chainlink CCIP. The `IBridgeAdapter` interface maps to CCIP's Router → OnRamp → DON → OffRamp flow.*

The resolution broadcast:

- Originates on the home chain (the bridge dispatcher composes it after watching `MarketResolved`).
- Is sent via `IBridgeAdapter.sendResolution(toChainId, proxyAddress, marketId, outcome, proofUrl)`.
- The CCIP adapter translates this into a CCIP `router.ccipSend()` call, targeting the `MarketProxy` on the destination chain.
- CCIP's Decentralized Oracle Network (DON) delivers the message; CCIP's Risk Management Network (RMN) provides an independent verification layer (parallel to Justify's circuit breaker — see [06-security-and-risk-management.md](#)).
- The `MarketProxy` on the destination chain receives the message via its `ccipReceive()` implementation, verifies the sender is the allowlisted home chain / home market, and marks itself resolved.

**Why CCIP?** The same properties that make CCIP suitable for cross-chain token transfers make it suitable for resolution broadcasts:

- **Guaranteed delivery** (or explicit revert on the source if undeliverable).
- **Verifiable origin** (proxy can cryptographically verify the home chain sent it).
- **Auditable** (CCIP message IDs are on-chain, CCIP explorers show cross-chain flow).
- **Production-ready** across the target networks (Arc, Base, Ethereum, Polygon, BSC).

### 3.3 Chainlink as the committed price source (optional, future)

Chainlink Data Feeds (e.g. ETH/USD, BTC/USD) or Chainlink Functions (custom compute) can serve as the **resolution data source** for the CRE resolver:

- Instead of polling Coingecko, the CRE service reads the on-chain Chainlink price feed at the market's `resolutionAt` timestamp.
- This makes resolution **fully on-chain-verifiable**: the proof URL points to the Chainlink aggregator contract + block number, not an external API endpoint.

This is an **optional enhancement** (not MVP). The architecture supports it: the CRE resolver is data-source-agnostic; the `oracleProofUrl` field records whatever source was used.

**Summary:** Chainlink is the foundation of Justify's decentralized resolution architecture — CRE for the decision, CCIP for the propagation, optionally Data Feeds for the input.

---

## 4. Resolution flow

### 4.1 End-to-end sequence

Time: market.resolutionAt has passed, price condition met

1. CRE resolver (Go):
  - fetch price from API / Chainlink Feed
  - compare vs market rule → decide outcome (YES=0 / NO=1)
  - check isResolved(marketId) → false
  - sign + send: OracleResolver.resolve(marketId, outcome, proofUrl)
2. HOME chain (OracleResolver.sol):
  - verify msg.sender has RESOLVER\_ROLE
  - verify !resolutions[marketId].resolved
  - store: resolutions[marketId] = {resolved: true, outcome, proofUrl}
  - emit MarketResolved(marketId, outcome, proofUrl)
3. Bridge dispatcher (watches MarketResolved):
  - parse event: marketId=42, outcome=0, proofUrl="..."
  - lookup proxy chains for this market (Base, Polygon, BSC)
  - for each proxy chain:
    - IBridgeAdapter.sendResolution(toChainId, proxyAddress, 42, 0, proofUrl)
4. CCIP adapter (per destination):
  - compose CCIP message: data=abi.encode(marketId, outcome, proofUrl)
  - call router.ccipSend(destChainSelector, message)
  - pay LINK fee
  - emit ResolutionSent(messageId, toChain, marketId)
5. CCIP DON + RMN:
  - attest message
  - deliver to destination OffRamp
6. PROXY chain (MarketProxy.ccipReceive):
  - verify sender = allowlisted home chain / home market
  - verify !\_resolved (dedup by marketId)
  - store: \_resolved = true, \_outcome = outcome
  - emit ProxyResolved(marketId, outcome, proofUrl)
7. Users on proxy chain:
  - call MarketProxy.redeem(wrappedReceiptTokenId, amount)
  - contract burns wrapped receipt
  - if outcome matches receipt's position → transfer local collateral
  - else → no payout (losing outcome)

### 4.2 On-chain function signatures

```
// HOME chain – OracleResolver.sol
function resolve(uint256 marketId, uint8 winningOutcome, string calldata oracleProofUrl) external;
// gated: acl.hasRole(RESOLVER_ROLE, msg.sender)
```

```
// reverts AlreadyResolved if resolutions[marketId].resolved

event MarketResolved(uint256 indexed marketId, uint8 winningOutcome, string oracleProofUrl);

function isResolved(uint256 marketId) external view returns (bool);
```

```
// HOME chain – IBridgeAdapter (dispatcher calls this)
function sendResolution(
 uint256 toChainId,
 address proxyAddress,
 uint256 marketId,
 uint8 outcome,
 string calldata proofUrl
) external payable returns (bytes32 messageId);
```

```
// PROXY chain – MarketProxy.sol
function receiveResolution(uint256 marketId, uint8 outcome, string calldata proofUrl) internal;
// called by ccipReceive after sender verification
// marks _resolved = true, unlocks redeem()

function redeem(uint256 wrappedReceiptId, uint256 amount) external;
// requires _resolved == true
// burns wrapped receipt, pays out local collateral if outcome matches
```

## 5. Idempotency and safety

### 5.1 Home-chain deduplication

`OracleResolver.resolve()` is idempotent on-chain:

- The `resolutions[marketId].resolved` flag is checked before every write.
- A second call for the same `marketId` reverts with `AlreadyResolved()`.
- The CRE resolver can pre-check `isResolved(marketId)` to skip already-resolved markets in batch runs (no wasted gas).

### 5.2 Proxy-chain deduplication

Each `MarketProxy` tracks its own `_resolved` state per market:

- The first `receiveResolution(marketId, ...)` marks `_resolved = true`.
- Subsequent broadcasts for the same `marketId` are ignored (no revert; silent no-op).
- CCIP message IDs are logged in the `ProxyResolved` event for audit trails; the contract does not store them (resolution is final, no replay needed).

This protects against:

- Accidental double-send from the bridge dispatcher.
- CCIP retry/redelivery (unlikely but theoretically possible).

### 5.3 Proxy unavailability — pull-based redemption

**What happens if a proxy chain is down or the CCIP lane is paused at broadcast time?**

The resolution broadcast is **asynchronous and retrievable**:

- If the CCIP send fails (destination chain halted, gas spike, lane paused), the dispatcher logs the failure and retries later (exponential backoff).

- Users on the unreachable proxy chain **cannot redeem yet** (their `MarketProxy._resolved` is still false), but their wrapped receipts remain valid — no loss of funds.
- Once the lane reopens and the broadcast succeeds, redemption unlocks.

**Pull-based safety:** redemption is a **pull operation** initiated by the user (`MarketProxy.redeem()`), not a push from the home chain. The proxy never needs to "call home" to settle; it waits for the broadcast, then settles locally. This decouples settlement availability from real-time bridge liveness.

## 5.4 Circuit breaker does NOT block redemptions

From [06-security-and-risk-management.md](#):

*The circuit breaker can pause **new proxy intents** (new buys/sells) but NEVER blocks redemptions.*

Even if the circuit breaker is triggered (anomaly detection, bridge exploit), users can always redeem **already-resolved** positions. The `MarketProxy.redeem()` function is explicitly excluded from the pause gate.

Why: redemption is the safety valve. If a bridge is compromised, freezing user funds on the proxy chain makes the damage worse. Allow exit.

## 5.5 Misordered broadcasts

Markets resolve in wall-clock order (when their `resolutionAt` time passes), but CCIP delivery times vary per lane (Base may be 30s, BSC may be 2min). This is fine:

- Each `MarketProxy` resolves independently; there is no cross-market dependency.
- A market's resolution on proxy chain A does not block redemption of a different market on proxy chain B.
- Users see "Resolved — pending bridge confirmation" UI until their proxy receives the broadcast; then redemption unlocks.

# 6. Current state and future work

## 6.1 Current (MVP / Arc phase)

From [cre-resolver-plan.md](#) :

*Landmine 1 — there are TWO unconnected `resolve()`*

- `OracleResolver.resolve(id, outcome, proof)` → records + emits, gated by `RESOLVER_ROLE` . ← **CRE resolver calls this.**
- `PredictionMarket.resolve(outcome)` → moves the market state machine `Open`→`Closed`→`Resolved`, gated only by `onlyCreator` . **Different contract, different gate.**

*These are NOT linked. Calling `OracleResolver` does **not** advance the `PredictionMarket` state, so payout/redeem won't trigger.*

**Current scope (single-chain, Chainlink CRE track):**

- The CRE resolver Go service exists and calls `OracleResolver.resolve()` on the home chain (Arc testnet).
- The `MarketResolved` event is emitted on-chain; the transaction is visible in Arcscan (audit trail).
- **Gap:** resolution is recorded but not yet wired to `PredictionMarket.resolve()` , so redemption/payout does not trigger automatically.

This is **sufficient for the CRE Chainlink prize** (automated off-chain compute → on-chain write, meaningfully used). The full lifecycle is Phase 3 work.

## 6.2 Future work (bridge phase)

**To complete cross-chain resolution:**

1. **Wire `OracleResolver` → `PredictionMarket`** : when `OracleResolver.resolve()` is called, trigger `PredictionMarket.resolve()` or vice versa. One write, both states updated. Design options:

- `OracleResolver` holds a reference to `PredictionMarket` and calls it.
  - `PredictionMarket.resolve()` requires oracle confirmation (two-phase commit).
  - `MarketFactory` wires them at market creation.
2. **Implement the bridge dispatcher:** off-chain service (or keeper contract) that watches `MarketResolved` events on the home chain, looks up all proxy addresses for that market (from the backend's bridge registry), and calls `IBridgeAdapter.sendResolution(...)` for each.
  3. **Deploy MarketProxy contracts:** one per proxy chain per market (or a singleton proxy per chain that handles multiple markets). Implement `ccipReceive()` to accept resolution broadcasts and unlock `redeem()`.
  4. **CCIP adapter production config:** fund LINK, allowlist lanes, set gas limits per [04-per-chain-bridge-playbook.md](#).
  5. **End-to-end test:** create a market on Arc, proxy it to Base, trade on both, resolve via CRE on Arc, verify the broadcast reaches Base, redeem on both chains.

### 6.3 Honest scope note

**The bridge phase is designed but not yet built.** The contracts exist ( `OracleResolver` , the `IBridgeAdapter` interface is specced, `MarketProxy` is designed), the CRE resolver exists and works on the home chain, and the Chainlink CCIP adapter design is documented. The **integration** (wiring resolution to payout, multi-chain broadcast, proxy redemption) is Phase 3 work per the [whitepaper roadmap](#) §8.

This document defines the **target architecture** so that when the bridge phase is built, the seams are already frozen and the contracts can be written against this spec.

---

## References

- [Justify whitepaper §5.3](#) — resolution Phase 1 (manual) → Phase 2 (CRE automated) → Phase 3 (bridge propagated).
- [CRE resolver plan](#) — the existing Go service, `RESOLVER_ROLE` , oracle key, `OracleResolver.resolve()` signature.
- [OracleResolver.sol](#) — the on-chain contract.
- [02-chainlink-ccip-adapter.md](#) — CCIP as the bridge transport.
- [03-marketproxy-and-home-market.md](#) — proxy contract design, wrapped receipts, redemption flow.
- [06-security-and-risk-management.md](#) — circuit breaker, exposure caps, redemption always allowed.

---

**Status:** Designed (Phase 3). The CRE resolver (Chainlink track) is built and working on the home chain; cross-chain propagation is future work. This document is the integration contract for that phase.

# Security and Risk Management — Justify Bridge Layer

**Document:** 06-security-and-risk-management.md

**Status:** Designed (Phase 3 — bridge-proxied markets)

**Related:** [Whitepaper §5.2 & §7](#) | [01-bridge-adapter-interface.md](#) | [02-chainlink-ccip-adapter.md](#) | [03-marketproxy-and-home-market.md](#) | [05-cross-chain-resolution.md](#)

## 1. Trust model and attack surface

### 1.1 The fundamental dependency stack

A cross-chain position in Justify depends on the integrity of **five layers**:

|                                                   |                                              |
|---------------------------------------------------|----------------------------------------------|
| Home Chain Safety & Liveness                      | ← Must produce blocks, finalize, not reorg   |
| Home Market Contracts (MarketAMM, OracleResolver) | ← Must price/settle correctly                |
| Bridge Adapter Transport (CCIP DON, LayerZero)    | ← Must deliver messages honestly, in order   |
| MarketProxy Contract (lock/mint/redemption logic) | ← Must escrow collateral, honor settlements  |
| Proxy Chain Safety & Liveness<br>reorg            | ← Must finalize user transactions, not reorg |

**Whitepaper §5.2 trust model:** "A proxy position is as safe as the **weakest of** (home chain, bridge adapter)." This is the honest statement: cross-chain positions inherit the risk of **both** the home settlement layer **and** the bridge transport. Where a single-chain position depends on one chain's liveness, a proxied position depends on two chains + the bridge integrity.

### 1.2 Quantifying the added attack surface

| Attack vector                    | Single-chain market      | Bridge-proxied market                                                                                    |
|----------------------------------|--------------------------|----------------------------------------------------------------------------------------------------------|
| Chain liveness failure           | 1 chain (home)           | 2 chains (home + proxy); if either halts, new positions freeze (but redemptions still possible via pull) |
| Chain reorg / finality violation | 1 chain (home)           | 2 chains; reorg on either can cause message replay or state divergence                                   |
| Contract bug                     | Home market contracts    | Home market + proxy contracts + bridge adapter contracts                                                 |
| Oracle manipulation              | Home chain resolver only | Home chain resolver (resolution propagates outward, so no new attack but inherited risk)                 |
| Censorship / MEV                 | Home chain validators    | Home chain + proxy chain + bridge relay operators (3 censorship points)                                  |
| Bridge transport failure         | N/A                      | Message loss, replay, reordering, or malicious message injection by compromised bridge                   |
| Collateral custody               | 1 pool (home AMM)        | Collateral escrowed on proxy + collateral in home AMM (larger blast radius if proxy is compromised)      |

**The honest assessment:** Bridge-proxied markets trade **distribution** (users trade from any chain) for **complexity** (more failure modes). The safety architecture below mitigates the added risk but does not eliminate it.

## 2. Threat catalog with mitigations

| Threat                                        | Attack scenario                                                                                                                                                        | Mitigation                                                                                                                                                                                                                |
|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Malicious bridge message</b>               | A compromised or malicious bridge adapter sends a fabricated <code>FillConfirmation</code> to mint unbacked wrapped receipts on a proxy, draining the collateral pool. | <b>Adapter allowlist per route</b> (§6). Only governance-approved adapters can send messages. CCIP's Risk Management Network (§3) provides a second blessing layer for CCIP routes.                                       |
| <b>Message replay attack</b>                  | An attacker re-sends a valid historical message (e.g., a <code>BuyIntent</code> ) multiple times to drain collateral or double-mint receipts.                          | <b>On-chain idempotency</b> (§2.4 of doc 01). Each message ID is hashed and stored in <code>processedMessages</code> mapping; duplicate IDs revert. Per-trader nonces enforce ordering.                                   |
| <b>Source chain reorg</b>                     | Proxy chain reorgs after a <code>BuyIntent</code> is sent; the intent never actually happened on-chain, but the home market already filled it.                         | <b>Finality delay on proxy.</b> Proxy only sends intents after the transaction reaches finality (e.g., 64 blocks on Ethereum). CCIP enforces finality thresholds before accepting messages.                               |
| <b>Home chain reorg</b>                       | Home market fills an intent, sends <code>FillConfirmation</code> , then reorgs; the fill is undone but the proxy already minted receipts.                              | <b>Finality delay on home.</b> Proxies only accept confirmations after home-chain finality. CCIP's OffRamp enforces this. Manual disaster recovery if a deep reorg occurs (pause, reconcile, redeploy if necessary).      |
| <b>Bridge adapter bug / compromise</b>        | A bug in the CCIP adapter or LayerZero adapter allows message forgery or message loss.                                                                                 | <b>Per-route exposure caps</b> (§6). Each route has a max outstanding collateral cap; once hit, new intents revert. Limits blast radius. + <b>Adapter allowlist:</b> only audited, governance-approved adapters.          |
| <b>Oracle / resolution manipulation</b>       | A malicious resolver settles a market incorrectly on the home chain; the bad resolution propagates to all proxies.                                                     | <b>Inherited from home chain OracleResolver.</b> Bridge adds no new attack surface here (resolution is home-authoritative). Future: decentralized resolution via CRE (doc 05) + dispute escalation removes single point.  |
| <b>Liquidity drain via a bad route</b>        | An attacker exploits a low-security bridge route (e.g., a newly added, poorly audited adapter) to drain proxy collateral into the home market, then extracts it.       | <b>Per-route exposure caps</b> (§6). Each route is capped independently; a bad route can only drain up to its cap, not the entire protocol. + <b>Allowlist:</b> only vetted adapters per route.                           |
| <b>Griefing via stuck intents</b>             | An attacker sends many small intents that intentionally fail on the home market (e.g., max slippage = 0), locking collateral on the proxy and forcing refunds.         | <b>Refund gas cost borne by attacker.</b> Intentionally failing intents still pay bridge fees. Protocol can set a minimum intent size to make griefing uneconomical. Circuit breaker trips on anomalous refund rate (§4). |
| <b>Front-running / MEV on bridge messages</b> | A validator or relayer observes a large <code>BuyIntent</code> in the bridge queue and front-runs it on the home AMM to move the price.                                | <b>Slippage bounds in intent.</b> User's <code>maxSlippage</code> parameter enforces bounded fill or refund; attacker cannot force a worse-than-quoted fill.                                                              |

|                                                                |                                                                                                      |                                                                                                                                                                                                                             |
|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                                |                                                                                                      | Home AMM is CPMM (no order book), so front-running is limited to sandwich.                                                                                                                                                  |
| <b>Circuit breaker manipulation</b>                            | An attacker triggers the circuit breaker (§4) to halt new proxy intents, grieving users.             | <b>High anomaly thresholds.</b> Circuit breaker only trips on statistically significant deviations (e.g., >5% of intents refunded in 1 hour). Governance can tune thresholds per route.                                     |
| <b>Collateral token compromise (proxy chain)</b>               | Proxy chain's USDC contract is compromised or frozen; escrowed collateral is lost.                   | <b>Chain-canonical stablecoins only</b> (whitepaper §7). Justify only uses Circle USDC (where available) or audited, widely adopted stablecoins. + Per-route exposure caps bound loss.                                      |
| <b>Smart contract upgrade attack (malicious proxy upgrade)</b> | If proxies were upgradeable, a malicious admin could upgrade the logic to steal escrowed collateral. | <b>No upgradeable contracts</b> (whitepaper §7, injected-dependency pattern). Proxies are immutable; a bad parameter is fixed by redeploying a <i>new</i> proxy, not by upgrading. Admin keys cannot mutate deployed logic. |

### 3. The Chainlink Risk Management Network parallel — defense-in-depth inspiration

#### 3.1 Chainlink CCIP's Risk Management Network (RMN)

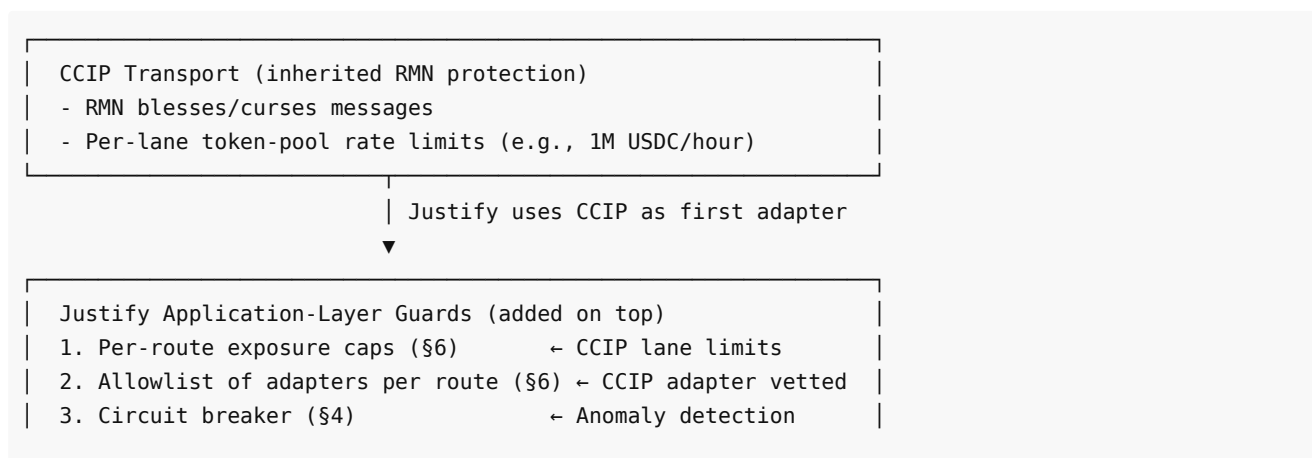
Chainlink CCIP has an **independent Risk Management Network** — a separate set of nodes that monitors all cross-chain messages and provides a second validation layer:

- **Blessing:** The RMN independently verifies that a message from an OnRamp is legitimate (not forged, not replayed, within rate limits). The OffRamp only executes messages that are *both* delivered by the Decentralized Oracle Network (DON) *and* blessed by the RMN.
- **Cursing:** If the RMN detects anomalies (e.g., a DON delivering messages that violate lane rate limits or finality rules), it can **curse** a lane, halting all message delivery on that route until governance intervenes.
- **Per-lane rate limits:** CCIP enforces token-transfer caps per lane (e.g., max 1M USDC/hour on Base → Ethereum). The RMN monitors these; exceeding the cap triggers a curse.

**The result:** Even if the CCIP DON is compromised, the RMN acts as a secondary check. An attacker must compromise *both* the DON and the RMN to inject a malicious message — defense-in-depth.

#### 3.2 Justify's bridge safety architecture — RMN-inspired, application-layer

Justify's bridge layer is **explicitly inspired by Chainlink CCIP's defense-in-depth model (Risk Management Network + per-lane rate limits)**. Where CCIP operates at the transport layer (blessing/cursing messages), Justify adds **application-layer guards** on top:



By using CCIP as the transport, Justify **INHERITS** the RMN's protection (messages already blessed by an independent network), **THEN ADDS** its own application-specific defenses:

1. **Per-route exposure caps** (analogous to CCIP lane rate limits, but at the Justify protocol layer).
2. **Adapter allowlists** (only governance-approved adapters can route messages into Justify markets).
3. **Circuit breaker** (pauses new proxy intents on anomaly detection, never blocks redemptions).
4. **On-chain idempotency and nonce ordering** (prevents replays and enforces per-trader ordering).

**The philosophy:** Trust the bridge transport (CCIP, LayerZero), but verify at the application layer. Even if a bridge adapter is compromised, Justify's caps and circuit breaker bound the blast radius.

## 4. The circuit breaker — pause new intents, never block redemptions

### 4.1 Design invariant

**Whitepaper §7:** "A circuit-breaker pausing new proxy intents but NEVER blocking redemptions."

**The non-negotiable rule:** Users can **always exit their positions**, even if the bridge is compromised or the circuit breaker has tripped. The circuit breaker halts new *entries* (new buy/sell intents from proxies) but never stops redemptions (unwinding positions after settlement).

### 4.2 What trips the circuit breaker

The circuit breaker monitors **anomaly signals** per route:

| Anomaly threshold                                             | Action                                  | Rationale                                                                                                                        |
|---------------------------------------------------------------|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Refund rate &gt; 5% of intents in 1-hour window</b>        | Trip breaker for that route             | Normal refund rate due to slippage is ~0.1–0.5%. A spike to >5% suggests AMM manipulation, bridge message corruption, or attack. |
| <b>Outstanding collateral &gt; route exposure cap</b>         | Revert new intents (automatic, no trip) | Hard cap per route (§6); once hit, no new intents until outstanding collateral drops below cap.                                  |
| <b>Home market fails to confirm &gt; 50% of intents in 1h</b> | Trip breaker for that route             | Suggests home-chain liveness failure or bridge message loss.                                                                     |
| <b>Manual governance intervention</b>                         | Trip breaker globally or per route      | Emergency response to detected compromise or anomaly not covered by automated thresholds.                                        |

**When tripped:** The proxy's `buy()` and `sell()` functions revert with `CircuitBreakerTripped()` error. Existing positions are unaffected; users can still redeem after settlement (redemption is local to the proxy, no bridge message required in the pull-based model — see §4.3).

### 4.3 Redemption invariant — always exit-able

**Two redemption paths:**

1. **Home-chain redemption:** Users with native home-market positions (no proxy) redeem directly on the home chain after `OracleResolver.resolve()` — no bridge dependency.
2. **Proxy-chain redemption:** Users with wrapped receipts on a proxy redeem *after* the home-market resolution is bridged to the proxy (doc 05). If the bridge is down, the proxy has a **pull-based redemption fallback**:

```
// Fallback: user provides Merkle proof of home-market resolution
// (sourced from home chain via off-chain indexer or explorer)
function redeemWithProof(
 uint256 marketId,
 uint8 winningOutcome,
 bytes32[] calldata merkleProof
) external {
 // Verify proof against root stored by governance
 require(verifyResolutionProof(marketId, winningOutcome, merkleProof), "Invalid proof");
 // Burn wrapped receipt, release collateral
 _redeem(msg.sender, marketId, winningOutcome);
}
```

**Result:** Even if the bridge is permanently compromised and the circuit breaker trips forever, users can exit via the pull-based proof path. Redemptions are **never blocked**.

## 4.4 Recovery and reset

After the circuit breaker trips:

1. **Diagnosis:** Governance investigates the anomaly (on-chain forensics, bridge adapter audit, home-market state check).
2. **Remediation:** If a bug is found, deploy a patched adapter or proxy; if an attack is detected, blacklist the malicious adapter.
3. **Reset:** Governance calls `CircuitBreaker.reset(routeId)` to re-enable new intents on that route.
4. **Post-mortem:** Document the incident, tune thresholds if false positive, publish transparency report.

# 5. Governance and access control

## 5.1 Role model (consistent with AccessControl.sol)

Justify's contracts use OpenZeppelin's `AccessControl` for role-based permissions. The bridge layer extends this:

| Role               | Powers                                                                                                     | Holder(s)                                                   |
|--------------------|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| DEFAULT_ADMIN_ROLE | Can grant/revoke all roles; top-level governance.                                                          | Deployer (initially), then multisig or DAO (future).        |
| RESOLVER_ROLE      | Can call <code>OracleResolver.resolve()</code> to settle markets on home chain.                            | Authorized resolver service (manual in MVP, CRE in future). |
| FACTORY_ROLE       | Can call <code>MarketFactory.createMarket()</code> to deploy new markets.                                  | Backend service (validates market-creation requests).       |
| BRIDGE_ADMIN_ROLE  | Can allowlist/denylist bridge adapters per route; set per-route exposure caps; trip/reset circuit breaker. | Governance multisig (3/5 or DAO).                           |
| TREASURY_ROLE      | Can withdraw fees from <code>FeeTreasury</code> .                                                          | Protocol treasury multisig.                                 |

**New role for bridge phase:** `BRIDGE_ADMIN_ROLE` controls the bridge safety parameters. This role is separate from `RESOLVER_ROLE` and `FACTORY_ROLE` to enforce separation of concerns (bridge operations vs. market operations).

## 5.2 Adapter allowlist and per-route registration

Each bridge route (e.g., Arc → Base via CCIP, Base → Polygon via LayerZero) requires explicit governance approval:

```
// In BridgeRegistry (deployed once per chain)
struct RouteConfig {
```

```

 address adapter; // IBridgeAdapter implementation
 uint256 exposureCap; // Max outstanding collateral for this route (USDC units)
 bool enabled; // Circuit breaker state
 uint64 homeChainSelector; // CCIP chain selector for home chain
 uint64 proxyChainSelector; // CCIP chain selector for proxy chain
}

mapping(bytes32 => RouteConfig) public routes;

// Only BRIDGE_ADMIN_ROLE can call
function registerRoute(
 bytes32 routeId,
 address adapter,
 uint256 exposureCap,
 uint64 homeChainSelector,
 uint64 proxyChainSelector
) external onlyRole(BRIDGE_ADMIN_ROLE) {
 require(adapter != address(0), "Invalid adapter");
 require(exposureCap > 0, "Cap must be positive");
 routes[routeId] = RouteConfig({
 adapter: adapter,
 exposureCap: exposureCap,
 enabled: true,
 homeChainSelector: homeChainSelector,
 proxyChainSelector: proxyChainSelector
 });
 emit RouteRegistered(routeId, adapter, exposureCap);
}

function disableRoute(bytes32 routeId) external onlyRole(BRIDGE_ADMIN_ROLE) {
 routes[routeId].enabled = false;
 emit RouteDisabled(routeId);
}

```

#### Process for adding a new route:

1. Governance proposal: "Add Base → Polygon route via LayerZero adapter X at 500k USDC exposure cap."
2. Audit the adapter contract + integration test + testnet trial.
3. Governance multisig calls `BridgeRegistry.registerRoute()`.
4. Route is live; proxies on Polygon can now forward intents to Base home markets.

### 5.3 Upgrade and redeployment philosophy (whitepaper §7)

**No upgradeable god-contracts.** Justify follows the **injected-dependency pattern**:

- Contracts are **immutable** (no `delegatecall` proxies, no admin functions that mutate core logic).
- External dependencies (collateral token, bridge adapter, fee treasury) are **constructor parameters**, not storage slots an admin can rewrite.
- If a parameter is wrong (e.g., a bad adapter), the fix is to **redploy a new contract** with the correct parameter, not to upgrade the existing one.

**Why:** Upgradeable contracts create a trust assumption (users trust the admin key not to rug). Immutable contracts + redeployment removes that trust vector. The cost is operational (redployment requires migrating state or deprecating the old contract), but the security gain is worth it.

#### For bridges specifically:

- A bad `MarketProxy` is deprecated (circuit breaker tripped, users migrate to a new proxy with a patched adapter).

- A bad `IBridgeAdapter` is removed from the allowlist; a new audited adapter is registered.
- The home market is never upgraded (resolution is final; no do-overs). A bug in the home market is a protocol-level incident requiring governance intervention (e.g., deploy a replacement market, compensate affected users via treasury).

---

## 6. Per-route exposure caps — bounding the blast radius

### 6.1 The threat model

Each bridge route is an **independent attack surface**. A newly added route with a less-audited adapter (e.g., a new LayerZero-class bridge) carries higher risk than a mature CCIP route. Without caps, a compromised route could drain the entire protocol's collateral.

**Per-route exposure caps bound the blast radius:** if a route's adapter is compromised and fabricates a malicious message, it can only extract collateral up to that route's cap, not the entire protocol TVL.

### 6.2 How exposure is tracked

Each `MarketProxy` tracks **outstanding collateral** for its route:

```
// In MarketProxy
uint256 public outstandingCollateral; // Total collateral locked in pending/filled intents

function buy(uint8 outcomeIndex, uint256 collateralIn, uint256 maxSlippage) external {
 // Check exposure cap (BridgeRegistry enforces this)
 require(
 outstandingCollateral + collateralIn <= bridgeRegistry.getExposureCap(routeId),
 "Route exposure cap exceeded"
);

 // Lock collateral, increment outstanding
 collateral.transferFrom(msg.sender, address(this), collateralIn);
 outstandingCollateral += collateralIn;

 // Send BuyIntent to home market
 bytes32 messageId = bridgeAdapter.sendMessage(...);
 emit IntentSent(messageId, outcomeIndex, collateralIn);
}

function _onFillConfirmation(FillConfirmation memory fill) internal {
 // Mint wrapped receipt, decrement outstanding
 wrappedReceipts[fill.trader][fill.outcomeIndex] += fill.sharesOut;
 outstandingCollateral -= fill.collateralIn; // (collateral now "converted" to shares)
 emit Filled(fill.trader, fill.sharesOut);
}

function _onRefund(RefundNotice memory refund) internal {
 // Unlock collateral, decrement outstanding
 collateral.transfer(refund.trader, refund.refundAmount);
 outstandingCollateral -= refund.refundAmount;
 emit Refunded(refund.trader, refund.refundAmount);
}
```

**Outstanding collateral decreases** when:

- A fill confirmation arrives (collateral converted to wrapped receipts).

- A refund notice arrives (collateral unlocked).
- A redemption happens (wrapped receipts burned, collateral paid out).

**Outstanding collateral increases** only on new intents (buy/sell).

### 6.3 Suggested parameters

| Route                         | Exposure cap (USDC) | Rationale                                                                                                                      |
|-------------------------------|---------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>Arc → Base (CCIP)</b>      | 5M                  | Highest security (CCIP + RMN); largest expected volume; Arc as home for stablecoin-native UX.                                  |
| <b>Base → Ethereum (CCIP)</b> | 10M                 | Ethereum is deepest liquidity; CCIP + RMN protection; high cap justified.                                                      |
| <b>Polygon → Base (CCIP)</b>  | 2M                  | Polygon has high retail volume; CCIP route; moderate cap.                                                                      |
| <b>BSC → Base (LayerZero)</b> | 500k                | BSC has large user base but LayerZero adapter is newer; conservative cap until proven. Raise cap after 3 months incident-free. |
| <b>Testnet routes</b>         | 100k (testnet USDC) | Low cap for testing; unlimited upside risk is acceptable on testnet.                                                           |

**Governance can adjust caps** via `BridgeRegistry.setExposureCap(routeId, newCap)` (requires `BRIDGE_ADMIN_ROLE` ). Caps should be raised gradually as routes prove reliability.

### 6.4 Relation to CCIP lane rate limits

Chainlink CCIP enforces **per-lane token-pool rate limits** (e.g., max 1M USDC/hour Base → Ethereum). Justify's per-route exposure caps are **complementary, not redundant**:

- **CCIP lane limit:** Bounds the *flow rate* (tokens/hour) across the bridge transport. Enforced by CCIP's token pools + RMN.
- **Justify exposure cap:** Bounds the *total outstanding* collateral locked in a route at any time. Enforced by Justify's `MarketProxy` .

**Example:**

- CCIP lane limit: 1M USDC/hour Base → Ethereum.
- Justify exposure cap: 10M USDC Base → Ethereum.
- Scenario: 10M USDC worth of intents are sent in the first hour (allowed by Justify cap), but CCIP lane limit only allows 1M to flow in that hour. Result: intents queue up; CCIP processes 1M/hour until all 10M are delivered. The exposure cap still binds total risk; the lane limit throttles the rate.

**The two limits work together:** CCIP lane limits prevent denial-of-service (flooding the bridge), Justify exposure caps prevent over-exposure to a single route's risk.

## 7. Operational runbook — monitoring, incident response, recovery

### 7.1 Monitoring signals (off-chain indexer / alerting service)

| Signal                                 | Alert threshold                        | Action                                                    |
|----------------------------------------|----------------------------------------|-----------------------------------------------------------|
| <b>Refund rate anomaly</b>             | >5% refunds in 1-hour window per route | Page on-call; investigate home-market state + bridge logs |
| <b>Outstanding collateral near cap</b> | >90% of exposure cap                   | Notify governance; consider raising cap or throttling     |

|                                              |                                                       |                                                             |
|----------------------------------------------|-------------------------------------------------------|-------------------------------------------------------------|
| <b>Intent confirmation latency</b>           | >10 minutes (CCIP normal: 2-5 min)                    | Check bridge adapter health; alert bridge provider          |
| <b>Unprocessed intents (stuck messages)</b>  | Intent sent >1 hour ago, no confirmation              | Manual replay via bridge explorer / adapter debug tool      |
| <b>Circuit breaker trip</b>                  | Any route disabled                                    | Page on-call; diagnose, remediate, document                 |
| <b>Home-market resolution not propagated</b> | Resolution on home chain >2 hours, no proxy broadcast | Manually trigger resolution broadcast (doc 05)              |
| <b>Large single intent (whale alert)</b>     | Intent >100k USDC                                     | Monitor for front-running / MEV; normal operation, log only |

## 7.2 Incident response — trip the breaker, diagnose, recover

### Phase 1: Detection and containment (minutes)

1. Automated monitoring detects anomaly (e.g., refund rate spike).
2. Circuit breaker auto-trips (if threshold exceeded) OR on-call admin manually trips via `BridgeRegistry.disableRoute(routeId)`.
3. New intents halt; existing positions and redemptions unaffected.

### Phase 2: Diagnosis (hours)

4. On-call team pulls logs from:
  - Proxy contract events ( `IntentSent` , `Filled` , `Refunded` ).
  - Bridge adapter (CCIP Explorer, LayerZero Scan).
  - Home market events ( `Trade` , `Resolved` ).
5. Identify root cause:
  - Bridge message corruption? → Adapter bug or transport issue.
  - Home AMM manipulation? → Front-running, liquidity drain, price oracle failure.
  - Malicious actor? → Trace attacker address, freeze if possible (collateral still escrowed).

### Phase 3: Remediation (days)

6. If adapter bug: deploy patched adapter, audit, register as new route, deprecate old route.
7. If transport issue: coordinate with bridge provider (Chainlink, LayerZero) for fix.
8. If attack: blacklist malicious adapter, compensate affected users from treasury if necessary.

### Phase 4: Recovery (days to weeks)

9. Once remediated, governance resets circuit breaker: `BridgeRegistry.enableRoute(routeId)` or `registerRoute(newRouteId, patchedAdapter, newCap)`.
10. Post-mortem published (transparency report): what happened, how fixed, parameter changes.
11. Resume normal operations; monitor for recurrence.

## 7.3 Recovery tools

### For stuck intents (bridge message lost or delayed):

- **Manual replay:** Governance can call `BridgeAdapter.replayMessage(messageId)` to resend a lost intent. Requires off-chain verification that the message was legitimately sent but not delivered.
- **Pull-based confirmation:** If the bridge is permanently down, users can submit a Merkle proof (sourced from home-chain state) to finalize their intent on the proxy (similar to redemption fallback in §4.3).

### For compromised route:

- **Route deprecation:** Disable the route, deploy a new proxy with a patched adapter. Users migrate positions to the new proxy via a "migration market" (governance can create a 1:1 swap market to move wrapped receipts from old proxy to new proxy without price risk).

**For home-market bug:**

- **No on-chain recovery** (contracts are immutable). Governance can:
  - Deploy a replacement market.
  - Use treasury to compensate affected users (if bug caused incorrect settlement).
  - Document as protocol incident; transparency report; adjust future market parameters.

---

## 8. Summary — defense-in-depth, RMN-inspired, exit-always

Justify's bridge security model is built on **six pillars**:

1. **Inherited CCIP RMN protection:** By using Chainlink CCIP as the primary transport, Justify inherits the Risk Management Network's blessing/cursing and per-lane rate limits — a proven defense-in-depth architecture (§3).
2. **Application-layer exposure caps:** Per-route caps bound the blast radius of a compromised adapter, independent of CCIP's lane limits (§6).
3. **Adapter allowlists:** Only governance-approved, audited adapters can route messages into Justify markets (§5).
4. **Circuit breaker:** Automated anomaly detection trips a breaker to halt new intents while preserving redemptions — the user's exit is never blocked (§4).
5. **Immutable contracts + injected dependencies:** No upgradeable god-contracts; a bad parameter is fixed by redeployment, not mutation — removes admin-key rug risk (§5.3).
6. **On-chain idempotency + nonce ordering:** Prevents replay attacks and enforces per-trader message ordering (doc 01, §4).

**The honest trade-off:** Cross-chain positions are "as safe as the weakest of (home chain, bridge adapter)" (§1). Justify's architecture does not eliminate bridge risk — it **bounds, monitors, and mitigates** it. Users who cannot tolerate bridge risk can trade natively on the home chain with zero bridge dependency.

**The non-negotiable invariant (whitepaper §7):** Users can **always exit their positions**, even if the bridge fails, the circuit breaker trips, or the protocol is in disaster-recovery mode. Redemptions are never blocked.

---

**Forward links:**

- [02-chainlink-ccip-adapter.md](#) — The CCIP adapter that provides RMN protection and lane rate limits at the transport layer.
- [03-marketproxy-and-home-market.md](#) — How `MarketProxy` enforces exposure caps and circuit breaker checks on buy/sell.
- [05-cross-chain-resolution.md](#) — Resolution propagation and the pull-based redemption fallback.

---

**Status:** Designed. Not deployed. This document defines the safety architecture for Phase 3 (bridge-proxied markets). The MVP (Phase 0) is single-chain; the multi-chain native phase (Phase 2) has no bridge dependency. This spec is the blueprint for when Justify bridges markets across Arc, Base, Ethereum, Polygon, and BSC.